

Deep Q-learning from Demonstrations

Todd Hester

Google DeepMind
toddhester@google.com

Matej Vecerik

Google DeepMind
matejvecerik@google.com

Olivier Pietquin

Google DeepMind
pietquin@google.com

Marc Lanctot

Google DeepMind
lanctot@google.com

Tom Schaul

Google DeepMind
schaul@google.com

Bilal Piot

Google DeepMind
piot@google.com

Dan Horgan

Google DeepMind
horgan@google.com

John Quan

Google DeepMind
johnquan@google.com

Andrew Sendonaris

Google DeepMind
sendos@yahoo.com

Ian Osband

Google DeepMind
iosband@google.com

Gabriel Dulac-Arnold

Google DeepMind
gabe@squirrelsoup.net

John Agapiou

Google DeepMind
jagapiou@google.com

Joel Z. Leibo

Google DeepMind
jzl@google.com

Audrunas Gruslys

Google DeepMind
audrunas@google.com

Abstract

Deep reinforcement learning (RL) has achieved several high profile successes in difficult decision-making problems. However, these algorithms typically require a huge amount of data before they reach reasonable performance. In fact, their performance during learning can be extremely poor. This may be acceptable for a simulator, but it severely limits the applicability of deep RL to many real-world tasks, where the agent must learn in the real environment. In this paper we study a setting where the agent may access data from previous control of the system. We present an algorithm, *Deep Q-learning from Demonstrations* (DQfD), that leverages small sets of demonstration data to massively accelerate the learning process even from relatively small amounts of demonstration data and is able to automatically assess the necessary ratio of demonstration data while learning thanks to a prioritized replay mechanism. DQfD works by combining temporal difference updates with supervised classification of the demonstrator's actions. We show that DQfD has better initial performance than Prioritized Dueling Double Deep Q-Networks (PDD DQN) as it starts with better scores on the first million steps on 41 of 42 games and on average it takes PDD DQN 83 million steps to catch up to DQfD's performance. DQfD learns to out-perform the best demonstration given in 14 of 42 games. In addition, DQfD leverages human demonstrations to achieve state-of-the-art results for 11 games. Finally, we show that DQfD performs better than three related algorithms for incorporating demonstration data into DQN.

Introduction

Over the past few years, there have been a number of successes in learning policies for sequential decision-making problems and control. Notable examples include deep model-free Q-learning for general Atari game-playing (Mnih et al. 2015), end-to-end policy search for control of robot motors (Levine et al. 2016), model predictive control with embeddings (Watter et al. 2015), and strategic policies that combined with search led

to defeating a top human expert at the game of Go (Silver et al. 2016). An important part of the success of these approaches has been to leverage the recent contributions to scalability and performance of deep learning (LeCun, Bengio, and Hinton 2015). The approach taken in (Mnih et al. 2015) builds a data set of previous experience using batch RL to train large convolutional neural networks in a supervised fashion from this data. By sampling from this data set rather than from current experience, the correlation in values from state distribution bias is mitigated, leading to good (in many cases, super-human) control policies.

It still remains difficult to apply these algorithms to real world settings such as data centers, autonomous vehicles (Hester and Stone 2013), helicopters (Abbeel et al. 2007), or recommendation systems (Shani, Heckerman, and Brafman 2005). Typically these algorithms learn good control policies only after many millions of steps of very poor performance in simulation. This situation is acceptable when there is a perfectly accurate simulator; however, many real world problems do not come with such a simulator. Instead, in these situations, the agent must learn in the real domain with real consequences for its actions, which requires that the agent have good on-line performance from the start of learning. While accurate simulators are difficult to find, most of these problems have data of the system operating under a previous controller (either human or machine) that performs reasonably well. In this work, we make use of this demonstration data to pre-train the agent so that it can perform well in the task from the start of learning, and then continue improving from its own self-generated data. Enabling learning in this framework opens up the possibility of applying RL to many real world problems where demonstration data is common but accurate simulators do not exist.

We propose a new deep reinforcement learning algorithm, *Deep Q-learning from Demonstrations* (DQfD), which leverages even very small amounts of demonstration data to massively accelerate learning. DQfD initially pre-trains solely on the demonstration data using a combination

从演示中进行深度 Q 学习

托德·赫斯特 Google DeepMind toddhester@google.com
Matej Vecerik Google DeepMind matejvecerik@google.com

比拉尔·皮奥特
谷歌 DeepMind piot@google.com

Dan Horgan 谷歌 DeepMind horgan@google.com

加布里埃尔·杜拉克·阿诺德
谷歌 DeepMind gabrie@squirrelsoup.net

约翰·阿加皮乌
谷歌 DeepMind jagapiou@google.com

Olivier Pietquin
谷歌 DeepMind pietquin@google.com

John Quan 谷歌 DeepMind johnquan@google.com

Joel Z. Leibo
谷歌 DeepMind jzl@google.com

Marc Lanctot
谷歌 DeepMind lanctot@google.com

安德鲁·森多纳里斯
(Andrew Sendonaris)
谷歌 DeepMind sendos@yahoo.com

Tom Schaul
谷歌 DeepMind schaul@google.com

伊恩·奥斯本
谷歌 DeepMind iosband@google.com

奥德鲁纳斯·格鲁斯利斯
谷歌 DeepMind audrunas@google.com

抽象的

深度强化学习 (RL) 在困难的决策问题上取得了多项引人注目的成功。然而，这些算法通常需要大量数据才能达到合理的性能。事实上，他们在学习过程中的表现可能非常差。这对于模拟器来说可能是可以接受的，但它严重限制了深度强化学习在许多现实世界任务中的适用性，在这些任务中，代理必须在真实环境中学习。在本文中，我们研究了一种设置，在该设置中，代理可以访问系统先前控制的数据。我们提出了一种算法，*Deep Q-learning from Demonstrations* (DQfD)，它利用少量的演示数据来大幅加速学习过程，即使是相对少量的演示数据，并且由于优先重播机制，能够在自动评估演示数据的必要比例。DQfD 的工作原理是将时间差更新与演示者行为的监督分类相结合。我们表明，DQfD 比优先决斗双深度 Q 网络 (PDD DQN) 具有更好的初始性能，因为它在 42 场比赛中的 41 场比赛中的前 100 万步中得分更高，平均需要 PDD DQN 8300 万步才能赶上 DQfD 的性能。DQfD 学会了超越 42 场比赛中 14 场的最佳演示。此外，DQfD 利用人体演示在 11 场比赛中取得了最先进的结果。最后，我们表明，在将演示数据合并到 DQN 中，DQfD 的性能优于三种相关算法。

介绍

在过去的几年里，在连续决策问题和控制的学习策略方面取得了许多成功。著名的例子包括用于一般 Atari 游戏的深度无模型 Q 学习 (Mnih 等人，2015 年)、用于控制机器人电机的端到端策略搜索 (Levine 等人，2016 年)、带有嵌入的模型预测控制 (Watter 等人，2015 年) 以及搜索引导相结合的战略策略。

在围棋比赛中击败人类顶级专家 (Silver et al. 2016)。这些方法成功的一个重要部分是利用最近对深度学习的可扩展性和性能的贡献 (LeCun、Bengio 和 Hinton 2015)。(Mnih et al. 2015) 中采用的方法使用批量强化学习构建了一个以前经验的数据集，并根据这些数据以有监督的方式训练大型卷积神经网络。通过从该数据集而不是从当前经验中进行采样，可以减轻状态分布偏差的值的相关性，从而产生良好的（在许多情况下，超人类的）控制政策。

将这些算法应用到现实世界中仍然很困难，例如数据中心、自动驾驶汽车 (Hester 和 Stone 2013)、直升机 (Abbeel 等人 2007) 或推荐系统 (Shani、Heckerman 和 Brafman 2005)。通常，这些算法只有在模拟中表现非常差的数百万步之后才能学习良好的控制策略。当有一个完全准确的模拟器时，这种情况是可以接受的。然而，许多现实世界的问题并不伴随着这样的模拟器。相反，在这些情况下，智能体必须在真实领域中学习，并对其行为产生真实的后果，这要求智能体从学习一开始就具有良好的在线性能。虽然准确的模拟器很难找到，但大多数问题都有系统在先前的控制器（人或机器）下运行的数据，该控制器运行得相当好。在这项工作中，我们利用这些演示数据来预训练代理，使其从学习开始就能够在任务中表现良好，然后从自己生成的数据中继续改进。在这个框架中实现学习开启了将强化学习应用于许多现实世界问题的可能性，在这些问题中演示数据很常见，但准确的模拟器并不存在。

我们提出了一种新的深度强化学习算法，*Deep Q-learning from Demonstrations* (DQfD)，它甚至利用非常少量的演示数据来大幅加速学习。DQfD 最初仅使用组合对演示数据进行预训练

of temporal difference (TD) and supervised losses. The supervised loss enables the algorithm to learn to imitate the demonstrator while the TD loss enables it to learn a self-consistent value function from which it can continue learning with RL. After pre-training, the agent starts interacting with the domain with its learned policy. The agent updates its network with a mix of demonstration and self-generated data. In practice, choosing the ratio between demonstration and self-generated data while learning is critical to improve the performance of the algorithm. One of our contributions is to use a prioritized replay mechanism (Schaul et al. 2016) to automatically control this ratio. DQfD out-performs pure reinforcement learning using Prioritized Dueling Double DQN (PDD DQN) (Schaul et al. 2016; van Hasselt, Guez, and Silver 2016; Wang et al. 2016) in 41 of 42 games on the first million steps, and on average it takes 83 million steps for PDD DQN to catch up to DQfD. In addition, DQfD out-performs pure imitation learning in mean score on 39 of 42 games and out-performs the best demonstration given in 14 of 42 games. DQfD leverages the human demonstrations to learn state-of-the-art policies on 11 of 42 games. Finally, we show that DQfD performs better than three related algorithms for incorporating demonstration data into DQN.

Background

We adopt the standard Markov Decision Process (MDP) formalism for this work (Sutton and Barto 1998). An MDP is defined by a tuple $\langle S, A, R, T, \gamma \rangle$, which consists of a set of states S , a set of actions A , a reward function $R(s, a)$, a transition function $T(s, a, s') = P(s'|s, a)$, and a discount factor γ . In each state $s \in S$, the agent takes an action $a \in A$. Upon taking this action, the agent receives a reward $R(s, a)$ and reaches a new state s' , determined from the probability distribution $P(s'|s, a)$. A policy π specifies for each state which action the agent will take. The goal of the agent is to find the policy π mapping states to actions that maximizes the expected discounted total reward over the agent's lifetime. The value $Q^\pi(s, a)$ of a given state-action pair (s, a) is an estimate of the expected future reward that can be obtained from (s, a) when following policy π . The optimal value function $Q^*(s, a)$ provides maximal values in all states and is determined by solving the Bellman equation:

$$Q^*(s, a) = \mathbb{E} \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \right].$$

The optimal policy π is then $\pi(s) = \operatorname{argmax}_{a \in A} Q^*(s, a)$. DQN (Mnih et al. 2015) approximates the value function $Q(s, a)$ with a deep neural network that outputs a set of action values $Q(s, \cdot; \theta)$ for a given state input s , where θ are the parameters of the network. There are two key components of DQN that make this work. First, it uses a separate target network that is copied every τ steps from the regular network so that the target Q-values are more stable. Second, the agent adds all of its experiences to a replay buffer $\mathcal{D}^{\text{replay}}$, which is then sampled uniformly to perform updates on the network.

The double Q-learning up-

date (van Hasselt, Guez, and Silver 2016) uses the current network to calculate the argmax over next state values and the target network for the value of that action. The double DQN loss is $J_{DDQ}(Q) = (R(s, a) + \gamma Q(s_{t+1}, a_{t+1}^{\max}; \theta') - Q(s, a; \theta))^2$, where θ' are the parameters of the target network, and $a_{t+1}^{\max} = \operatorname{argmax}_a Q(s_{t+1}, a; \theta)$. Separating the value functions used for these two variables reduces the upward bias that is created with regular Q-learning updates.

Prioritized experience replay (Schaul et al. 2016) modifies the DQN agent to sample more important transitions from its replay buffer more frequently. The probability of sampling a particular transition i is proportional to its priority, $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$, where the priority $p_i = |\delta_i| + \epsilon$, and δ_i is the last TD error calculated for this transition and ϵ is a small positive constant to ensure all transitions are sampled with some probability. To account for the change in the distribution, updates to the network are weighted with importance sampling weights, $w_i = (\frac{1}{N} \cdot \frac{1}{P(i)})^\beta$, where N is the size of the replay buffer and β controls the amount of importance sampling with no importance sampling when $\beta = 0$ and full importance sampling when $\beta = 1$. β is annealed linearly from β_0 to 1.

Related Work

Imitation learning is primarily concerned with matching the performance of the demonstrator. One popular algorithm, DAGGER (Ross, Gordon, and Bagnell 2011), iteratively produces new policies based on polling the expert policy outside its original state space, showing that this leads to no-regret over validation data in the online learning sense. DAGGER requires the expert to be available during training to provide additional feedback to the agent. In addition, it does not combine imitation with reinforcement learning, meaning it can never learn to improve beyond the expert as DQfD can.

Deeply AggreVaTeD (Sun et al. 2017) extends DAGGER to work with deep neural networks and continuous action spaces. Not only does it require an always available expert like DAGGER does, the expert must provide a value function in addition to actions. Similar to DAGGER, Deeply AggreVaTeD only does imitation learning and cannot learn to improve upon the expert.

Another popular paradigm is to setup a zero-sum game where the learner chooses a policy and the adversary chooses a reward function (Syed and Schapire 2007; Syed, Bowling, and Schapire 2008; Ho and Ermon 2016). Demonstrations have also been used for inverse optimal control in high-dimensional, continuous robotic control problems (Finn, Levine, and Abbeel 2016). However, these approaches only do imitation learning and do not allow for learning from task rewards.

Recently, demonstration data has been shown to help in difficult exploration problems in RL (Subramanian, Jr., and Thomaz 2016). There has also been recent interest in this combined imitation and RL problem. For example, the HAT algorithm transfers knowledge directly from human poli-

时间差异 (TD) 和监督损失。监督损失使算法能够学习模仿演示器，而 TD 损失使其能够学习自洽的价值函数，从中可以继续使用 RL 进行学习。预训练后，代理开始使用其学习的策略与域进行交互。该代理通过混合演示和自行生成的数据来更新其网络。在实践中，在学习时选择演示数据和自生成数据之间的比例对于提高算法的性能至关重要。我们的贡献之一是使用优先重放机制 (Schaul et al. 2016) 来自动控制这个比率。DQfD 在前 100 万步的 42 场比赛中，有 41 场比赛的表现优于使用优先决斗双 DQN (PDD DQN) 的纯强化学习 (Schaul 等人, 2016 年; van Hasselt, Guez 和 Silver, 2016 年; Wang 等人, 2016 年)，平均而言，PDD DQN 需要 8300 万步才能赶上 DQfD。此外，DQfD 在 42 场比赛中的 39 场比赛中的平均得分优于纯模仿学习，并且在 42 场比赛中的 14 场比赛中胜过给出的最佳演示。DQfD 利用真人演示来学习 42 场比赛中 11 场的最先进策略。最后，我们表明，在将演示数据合并到 DQN 中时，DQfD 的性能优于三种相关算法。

背景

我们在这项工作中采用标准马尔可夫决策过程 (MDP) 形式 (Sutton 和 Barto 1998)。MDP 由元组 $\langle S, A, R, T, \gamma \rangle$ 定义，它由一组状态 S 、一组动作 A 、奖励函数 $R(s, a)$ 、转换函数 $T(s, a, s') = P(s'|s, a)$ 和折扣因子 γ 组成。在每个状态 $s \in S$ 中，代理执行操作 $a \in A$ 。采取此操作后，代理会收到奖励 $R(s, a)$ 并达到根据概率分布 $P(s'|s, a)$ 确定的新状态 s' 。策略 π 指定代理将采取的每个状态的操作。代理的目标是找到将状态映射到操作的策略 π ，以最大化代理生命周期内的预期折扣总奖励。给定状态-动作对 (s, a) 的值 $Q^\pi(s, a)$ 是对遵循策略 π 时可以从 (s, a) 获得的预期未来奖励的估计。最优值函数 $Q^*(s, a)$ 提供所有状态下的最大值，并通过求解贝尔曼方程来确定：

$$Q^*(s, a) = \mathbb{E} \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \right].$$

最优策略 π 是 $\pi(s) = \operatorname{argmax}_{a \in A} Q^*(s, a)$ 。DQN (Mnih et al. 2015) 使用输出一组动作值 $Q(s, \cdot)$ 的深度神经网络来近似价值函数 $Q(s, a; \theta)$ 对于给定状态输入 s ，其中 θ 是网络的参数。DQN 有两个关键组件来实现这一点。首先，它使用一个单独的目标网络，每 τ 步骤从常规网络复制一次，以便目标 Q 值更加稳定。其次，代理将其所有经验添加到重播缓冲区 $\mathcal{D}^{\text{replay}}$ 中，然后对其进行统一采样以在网络上执行更新。

双Q学习

date (van Hasselt, Guez 和 Silver 2016) 使用当前网络来计算下一个状态值的 argmax 以及该操作值的目标网络。双 DQN 损失为

$$J_{DQ}(Q) = (R(s, a) + \gamma Q(s_{t+1}, a_{t+1}^{\max}; \theta') - Q(s, a; \theta))^2, \text{ 其中 } \theta' \text{ 是目标网络的参数, } a_{t+1}^{\max} = \operatorname{argmax}_a Q(s_{t+1}, a; \theta). \text{ 将用于这两个变量的值函数分开可以减少定期 } Q \text{ 学习更新所产生的向上偏差。}$$

优先体验重放 (Schaul et al. 2016) 修改了 DQN 代理，以更频繁地从重放缓冲区中采样更重要的转换。对特定转换 i 进行采样的概率与其优先级 $P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}$ 成正比，其中优先级 $p_i = |\delta_i| + \epsilon$ 和 δ_i 是为此转换计算的最后一个 TD 误差，而 ϵ 是一个小的正常数，以确保以某种概率对所有转换进行采样。为了考虑分布的变化，对网络的更新使用重要性采样权重 $w_i = (\frac{1}{N} \cdot \frac{1}{P(i)})^\beta$ 进行加权，其中 N 是重放缓冲区的大小，而 β 控制重要性采样的数量，当 $\beta = 0$ 时不进行重要性采样，而当 $\beta = 1$ 时进行完全重要性采样。 β 从 β_0 线性退火到 1。

相关工作

Imitation learning 主要关注匹配演示器的性能。DAGGER (Ross, Gordon 和 Bagnell 2011) 是一种流行的算法，它基于对原始状态空间之外的专家策略进行轮询，迭代地生成新策略，这表明这不会导致在线学习意义上的验证数据的遗憾。DAGGER 要求专家在培训期间随时为代理提供额外的反馈。此外，它没有将模仿与强化学习结合起来，这意味着它永远无法像 DQfD 那样学习超越专家。

Deeply AggreVaTeD (Sun et al. 2017) 将 DAGGER 扩展为与深度神经网络和连续动作空间一起使用。它不仅需要像 DAGGER 一样随时可用的专家，而且除了操作之外，专家还必须提供价值函数。与 DAGGER 类似，Deeply AggreVaTeD 只进行模仿学习，无法学习改进专家。

另一种流行的范式是设置一个零和博弈，其中学习者选择策略，对手选择奖励函数 (Syed 和 Schapire 2007; Syed, Bowling 和 Schapire 2008; Ho 和 Ermon 2016)。演示也被用于高维、连续机器人控制问题中的逆最优控制 (Finn, Levine 和 Abbeel 2016)。然而，这些方法只能进行模仿学习，不允许从任务奖励中学习。

最近，演示数据已被证明有助于解决 RL 中的困难探索问题 (Subramanian, Jr. 和 Thomaz 2016)。最近人们对这种模仿和强化学习相结合的问题也产生了兴趣。例如，HAT 算法直接从人类政治中转移知识。

cies (Taylor, Suay, and Chernova 2011). Follow-ups to this work showed how expert advice or demonstrations can be used to shape rewards in the RL problem (Brys et al. 2015; Suay et al. 2016). A different approach is to shape the policy that is used to sample experience (Cederborg et al. 2015), or to use policy iteration from demonstrations (Kim et al. 2013; Chemali and Lezaric 2015).

Our algorithm works in a scenario where rewards are given by the environment used by the demonstrator. This framework was appropriately called Reinforcement Learning with Expert Demonstrations (RLED) in (Piot, Geist, and Pietquin 2014a) and is also evaluated in (Kim et al. 2013; Chemali and Lezaric 2015). Our setup is similar to (Piot, Geist, and Pietquin 2014a) in that we combine TD and classification losses in a batch algorithm in a model-free setting; ours differs in that our agent is pre-trained on the demonstration data initially and the batch of self-generated data grows over time and is used as experience replay to train deep Q-networks. In addition, a prioritized replay mechanism is used to balance the amount of demonstration data in each mini-batch. (Piot, Geist, and Pietquin 2014b) present interesting results showing that adding a TD loss to the supervised classification loss improves imitation learning even when there are no rewards.

Another work that is similarly motivated to ours is (Schaal 1996). This work is focused on real world learning on robots, and thus is also concerned with on-line performance. Similar to our work, they pre-train the agent with demonstration data before letting it interact with the task. However, they do not use supervised learning to pre-train their algorithm, and are only able to find one case where pre-training helps learning on Cart-Pole.

In one-shot imitation learning (Duan et al. 2017), the agent is provided with an entire demonstration as input in addition to the current state. The demonstration specifies the goal state that is wanted, but from different initial conditions. The agent is trained with target actions from more demonstrations. This setup also uses demonstrations, but requires a distribution of tasks with different initial conditions and goal states, and the agent can never learn to improve upon the demonstrations.

AlphaGo (Silver et al. 2016) takes a similar approach to our work in pre-training from demonstration data before interacting with the real task. AlphaGo first trains a policy network from a dataset of 30 million expert actions, using supervised learning to predict the actions taken by experts. It then uses this as a starting point to apply policy gradient updates during self-play, combined with planning rollouts. Here, we do not have a model available for planning, so we focus on the model-free Q-learning case.

Human Experience Replay (HER) (Hosu and Rebedea 2016) is an algorithm in which the agent samples from a replay buffer that is mixed between agent and demonstration data, similar to our approach. Gains were only slightly better than a random agent, and were surpassed by their alternative approach, Human Checkpoint Replay, which requires the ability to set

the state of the environment. While their algorithm is similar in that it samples from both datasets, it does not pre-train the agent or use a supervised loss. Our results show higher scores over a larger variety of games, without requiring full access to the environment. Replay Buffer Spiking (RBS) (Lipton et al. 2016) is another similar approach where the DQN agent’s replay buffer is initialized with demonstration data, but they do not pre-train the agent for good initial performance or keep the demonstration data permanently.

The work that most closely relates to ours is a workshop paper presenting Accelerated DQN with Expert Trajectories (ADET) (Lakshminarayanan, Ozair, and Bengio 2016). They are also combining TD and classification losses in a deep Q-learning setup. They use a trained DQN agent to generate their demonstration data, which on most games is better than human data. It also guarantees that the policy used by the demonstrator can be represented by the apprenticeship agent as they are both using the same state input and network architecture. They use a cross-entropy classification loss rather than the large margin loss DQfD uses and they do not pre-train the agent to perform well from its first interactions with the environment.

Deep Q-Learning from Demonstrations

In many real-world settings of reinforcement learning, we have access to data of the system being operated by its previous controller, but we do not have access to an accurate simulator of the system. Therefore, we want the agent to learn as much as possible from the demonstration data before running on the real system. The goal of the pre-training phase is to learn to imitate the demonstrator with a value function that satisfies the Bellman equation so that it can be updated with TD updates once the agent starts interacting with the environment. During this pre-training phase, the agent samples mini-batches from the demonstration data and updates the network by applying four losses: the 1-step double Q-learning loss, an n-step double Q-learning loss, a supervised large margin classification loss, and an L2 regularization loss on the network weights and biases. The supervised loss is used for classification of the demonstrator’s actions, while the Q-learning loss ensures that the network satisfies the Bellman equation and can be used as a starting point for TD learning.

The supervised loss is critical for the pre-training to have any effect. Since the demonstration data is necessarily covering a narrow part of the state space and not taking all possible actions, many state-actions have never been taken and have no data to ground them to realistic values. If we were to pre-train the network with only Q-learning updates towards the max value of the next state, the network would update towards the highest of these ungrounded variables and the network would propagate these values throughout the Q function. We add a large margin classification loss (Piot, Geist, and Pietquin 2014a):

$$J_E(Q) = \max_{a \in A} [Q(s, a) + l(a_E, a)] - Q(s, a_E)$$

where a_E is the action the expert demonstrator took in state

(Taylor、Suay 和 Chernova 2011)。这项工作的后续工作展示了如何使用专家建议或演示来塑造 RL 问题的奖励 (Brys 等人, 2015 年; Suay 等人, 2016 年)。另一种不同的方法是制定用于采样经验的策略 (Cederborg et al. 2015), 或使用来自示范的政策迭代 (Kim et al. 2013; Chemali and Lezaric 2015)。

我们的算法适用于由演示者使用的环境给予奖励的场景。该框架在 (Piot、Geist 和 Pietquin 2014a) 中被适当地称为“专家演示强化学习”(RLED), 并且也在 (Kim 等人 2013; Chemali 和 Lezaric 2015) 中进行了评估。我们的设置类似于 (Piot、Geist 和 Pietquin 2014a), 我们在无模型设置的批处理算法中将 TD 和分类损失结合起来; 我们的不同之处在于, 我们的代理最初是在演示数据上进行预训练的, 并且一批自生成的数据随着时间的推移而增长, 并用作训练深度 Q 网络的经验回放。此外, 还使用优先重放机制来平衡每个小批量中的演示数据量。(Piot、Geist 和 Pietquin 2014b) 提出了有趣的结果, 表明即使没有奖励, 在监督分类损失中添加 TD 损失也能改善模仿学习。

另一项与我们的动机类似的工作是 (Schaal 1996)。这项工作的重点是机器人的现实世界学习, 因此也涉及在线性能。与我们的工作类似, 他们在让代理与任务交互之前使用演示数据对代理进行预训练。然而, 他们没有使用监督学习来预训练他们的算法, 并且只能找到一种预训练有助于 Cart-Pole 学习的情况。

在一次性模仿学习中 (Duan et al. 2017), 除了当前状态之外, 还为代理提供了完整的演示作为输入。该演示指定了所需的目标状态, 但初始条件不同。智能体接受来自更多演示的目标动作的训练。此设置也使用演示, 但需要分配具有不同初始条件和目标状态的任务, 并且代理永远无法学习改进演示。

AlphaGo (Silver et al. 2016) 在与实际任务交互之前, 采用了与我们的工作类似的方法, 从演示数据进行预训练。AlphaGo 首先根据 3000 万个专家动作的数据集训练策略网络, 使用监督学习来预测专家所采取的动作。然后, 它以此为起点, 在自我对弈期间应用策略梯度更新, 并结合规划部署。在这里, 我们没有可用于规划的模式, 因此我们重点关注无模型 Q 学习案例。

人类体验回放 (HER) (Hosu 和 Rebedea 2016) 是一种算法, 其中代理从混合代理和演示数据的重播缓冲区中进行采样, 类似于我们的方法。收益仅比随机代理稍好一些, 并且被他们的替代方法“人类检查点重放”超越, 该方法需要设置的能力

环境状况。虽然他们的算法相似, 都是从两个数据集中采样, 但它不会预先训练代理或使用监督损失。我们的结果显示, 无需完全访问环境, 即可在更多种类的游戏中获得更高的分数。重播缓冲区尖峰 (RBS) (Lipton 等人, 2016 年) 是另一种类似的方法, 其中 DQN 代理的重播缓冲区使用演示数据进行初始化, 但它们不会预先训练代理以获得良好的初始性能或永久保留演示数据。

与我们最密切相关的工作是一篇研讨会论文, 介绍了带有专家轨迹的加速 DQN (ADET) (Lakshminarayanan、Ozair 和 Bengio 2016)。他们还在深度 Q 学习设置中结合了 TD 和分类损失。他们使用训练有素的 DQN 代理来生成演示数据, 这在大多数游戏中都比人类数据更好。它还保证演示者使用的策略可以由学徒代理代表, 因为它们都使用相同的状态输入和网络架构。他们使用交叉熵分类损失, 而不是 DQN 使用的大边缘损失, 并且他们没有对代理进行预训练, 使其在与环境的第一次交互中表现良好。

从演示中进行深度 Q 学习

在强化学习的许多现实环境中, 我们可以访问由其先前控制器操作的系统的数据, 但我们无法访问系统的精确模拟器。因此, 我们希望代理在真实系统上运行之前尽可能多地从演示数据中学习。预训练阶段的目标是学习用满足贝尔曼方程的值函数来模仿演示器, 以便一旦代理开始与环境交互, 就可以使用 TD 更新来更新演示器。在预训练阶段, 智能体从演示数据中进行小批量采样, 并通过应用四种损失来更新网络: 1步双Q学习损失、n步双Q学习损失、有监督的大边缘分类损失以及网络权重和偏差的L2正则化损失。监督损失用于对演示者的行为进行分类, 而 Q 学习损失确保网络满足贝尔曼方程, 并且可以用作 TD 学习的起点。

监督损失对于预训练产生效果至关重要。由于演示数据必然覆盖状态空间的一小部分, 并且没有采取所有可能的行动, 因此许多状态行动从未被采取过, 也没有数据将它们与现实价值联系起来。如果我们只使用 Q 学习更新来预训练网络, 以达到下一个状态的最大值, 则网络将更新到这些无根据变量中的最高值, 并且网络将在整个 Q 函数中传播这些值。我们添加了较大的边缘分类损失 (Piot、Geist 和 Pietquin 2014a):

$$J_E(Q) = \max_{a \in A} [Q(s, a) + l(a_E, a)] - Q(s, a_E)$$

其中 a_E 是专家演示者在状态下采取的操作

s and $l(a_E, a)$ is a margin function that is 0 when $a = a_E$ and positive otherwise. This loss forces the values of the other actions to be at least a margin lower than the value of the demonstrator’s action. Adding this loss grounds the values of the unseen actions to reasonable values, and makes the greedy policy induced by the value function imitate the demonstrator. If the algorithm pre-trained with only this supervised loss, there would be nothing constraining the values between consecutive states and the Q-network would not satisfy the Bellman equation, which is required to improve the policy on-line with TD learning.

Adding n-step returns (with $n = 10$) helps propagate the values of the expert’s trajectory to all the earlier states, leading to better pre-training. The n-step return is:

$$r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a),$$

which we calculate using the forward view, similar to A3C (Mnih et al. 2016).

We also add an L2 regularization loss applied to the weights and biases of the network to help prevent it from over-fitting on the relatively small demonstration dataset. The overall loss used to update the network is a combination of all four losses:

$$J(Q) = J_{DQ}(Q) + \lambda_1 J_n(Q) + \lambda_2 J_E(Q) + \lambda_3 J_{L2}(Q).$$

The λ parameters control the weighting between the losses. We examine removing some of these losses in Section .

Once the pre-training phase is complete, the agent starts acting on the system, collecting self-generated data, and adding it to its replay buffer $\mathcal{D}^{replay}$. Data is added to the replay buffer until it is full, and then the agent starts overwriting old data in that buffer. However, the agent never over-writes the demonstration data. For proportional prioritized sampling, different small positive constants, ϵ_a and ϵ_d , are added to the priorities of the agent and demonstration transitions to control the relative sampling of demonstration versus agent data. All the losses are applied to the demonstration data in both phases, while the supervised loss is not applied to self-generated data ($\lambda_2 = 0$).

Overall, Deep Q-learning from Demonstration (DQfD) differs from PDD DQN in six key ways:

- **Demonstration data:** DQfD is given a set of demonstration data, which it retains in its replay buffer permanently.
- **Pre-training:** DQfD initially trains solely on the demonstration data before starting any interaction with the environment.
- **Supervised losses:** In addition to TD losses, a large margin supervised loss is applied that pushes the value of the demonstrator’s actions above the other action values (Piot, Geist, and Pietquin 2014a).
- **L2 Regularization losses:** The algorithm also adds L2 regularization losses on the network weights to prevent over-fitting on the demonstration data.
- **N-step TD losses:** The agent updates its Q-network with targets from a mix of 1-step and n-step returns.
- **Demonstration priority bonus:** The priorities of demonstration transitions are given a bonus of ϵ_d , to boost the frequency that they are sampled.

Pseudo-code is sketched in Algorithm 1. The behavior policy $\pi^{\epsilon Q_\theta}$ is ϵ -greedy with respect to Q_θ .

Algorithm 1 Deep Q-learning from Demonstrations.

- 1: Inputs: $\mathcal{D}^{replay}$: initialized with demonstration data set, θ : weights for initial behavior network (random), θ' : weights for target network (random), τ : frequency at which to update target net, k : number of pre-training gradient updates
 - 2: **for** steps $t \in \{1, 2, \dots, k\}$ **do**
 - 3: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
 - 4: Calculate loss $J(Q)$ using target network
 - 5: Perform a gradient descent step to update θ
 - 6: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
 - 7: **end for**
 - 8: **for** steps $t \in \{1, 2, \dots\}$ **do**
 - 9: Sample action from behavior policy $a \sim \pi^{\epsilon Q_\theta}$
 - 10: Play action a and observe (s', r) .
 - 11: Store (s, a, r, s') into $\mathcal{D}^{replay}$, overwriting oldest self-generated transition if over capacity
 - 12: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
 - 13: Calculate loss $J(Q)$ using target network
 - 14: Perform a gradient descent step to update θ
 - 15: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
 - 16: $s \leftarrow s'$
 - 17: **end for**
-

Experimental Setup

We evaluated DQfD on the Arcade Learning Environment (ALE) (Bellemare et al. 2013). ALE is a set of Atari games that are a standard benchmark for DQN and contains many games on which humans still perform better than the best learning agents. The agent plays the Atari games from a down-sampled 84x84 image of the game screen that has been converted to greyscale, and the agent stacks four of these frames together as its state. The agent must output one of 18 possible actions for each game. The agent applies a discount factor of 0.99 and all of its actions are repeated for four Atari frames. Each episode is initialized with up to 30 no-op actions to provide random starting positions. The scores reported are the scores in the Atari game, regardless of how the agent is representing reward internally.

For all of our experiments, we evaluated three different algorithms, each averaged across four trials:

- Full DQfD algorithm with human demonstrations
- PDD DQN learning without any demonstration data
- Supervised imitation from demonstration data without any environment interaction

We performed informal parameter tuning for all the algorithms on six Atari games and then used the same parameters for the entire set of games. The parameters used for the algorithms are shown in the appendix. Our coarse search over prioritization and n-step return parameters led to the same best parameters for DQfD and PDD DQN. PDD DQN

$s l(a_E, a)$ 是边际函数, 当 $a = a_E$ 时为 0, 否则为正值。这种损失迫使其他行动的价值至少比演示者行动的价值低一些。添加这种损失将看不见的动作的值建立在合理的值上, 并使价值函数引发的贪婪策略模仿示威者。如果算法仅使用这种监督损失进行预训练, 则连续状态之间的值将不受任何限制, 并且 Q 网络将无法满足 Bellman 方程, 而这是通过 TD 学习在线改进策略所必需的。

添加 n 步返回 ($n = 10$) 有助于将专家轨迹的值传播到所有早期状态, 从而实现更好的预训练。 n 步返回为:

$$r_t + \gamma r_{t+1} + \dots + \gamma^{n-1} r_{t+n-1} + \max_a \gamma^n Q(s_{t+n}, a),$$

我们使用前视来计算, 类似于 A3C (Mnih et al. 2016)。

我们还添加了应用于网络权重和偏差的 L2 正则化损失, 以帮助防止其在相对较小的演示数据集上过度拟合。用于更新网络的总体损失是所有四种损失的组合:

$$J(Q) = J_{DQ}(Q) + \lambda_1 J_n(Q) + \lambda_2 J_E(Q) + \lambda_3 J_{L2}(Q).$$

λ 参数控制损失之间的权重。我们在第 1 节中研究了如何消除其中一些损失。

预训练阶段完成后, 代理开始对系统进行操作, 收集自行生成的数据, 并将其添加到其重播缓冲区 $\mathcal{D}^{replay}$ 中。数据被添加到重播缓冲区直到填满, 然后代理开始覆盖该缓冲区中的旧数据。但是, 代理绝不会覆盖演示数据。对于比例优先采样, 不同的小正常数 ϵ_a 和 ϵ_d 被添加到代理和演示转换的优先级中, 以控制演示与代理数据的相对采样。所有损失都应用于两个阶段的演示数据, 而监督损失不应用于自行生成的数据 ($\lambda_2 = 0$)。

总体而言, 演示深度 Q 学习 (DQfD) 与 PDD DQN 在六个关键方面有所不同:

- 演示数据: DQfD 获得一组演示数据, 并将其永久保留在重播缓冲区中。
- 预训练: 在开始与环境进行任何交互之前, DQfD 最初仅根据演示数据进行训练。
- 监督损失: 除了 TD 损失之外, 还应用了较大裕度的监督损失, 将演示者行为的价值推至其他行为价值之上 (Piot, Geist 和 Pietquin 2014a)。
- L2 正则化损失: 该算法还在网络权重上添加了 L2 正则化损失, 以防止对演示数据的过度拟合。
- N 步 TD 损失: 代理使用 1 步和 n 步返回的混合目标更新其 Q 网络。
- 演示优先级奖励: 演示转换的优先级获得 ϵ_d 的奖励, 以提高采样频率。

伪代码如算法 1 所示。行为策略 $\pi^{\epsilon_{Q_\theta}}$ 相对于 Q_θ 是 ϵ 贪婪的。

Algorithm 1 Deep Q-learning from Demonstrations.

- 1: Inputs: $\mathcal{D}^{replay}$: initialized with demonstration data set, θ : weights for initial behavior network (random), θ' : weights for target network (random), τ : frequency at which to update target net, k : number of pre-training gradient updates
- 2: **for** steps $t \in \{1, 2, \dots, k\}$ **do**
- 3: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
- 4: Calculate loss $J(Q)$ using target network
- 5: Perform a gradient descent step to update θ
- 6: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
- 7: **end for**
- 8: **for** steps $t \in \{1, 2, \dots\}$ **do**
- 9: Sample action from behavior policy $a \sim \pi^{\epsilon_{Q_\theta}}$
- 10: Play action a and observe (s', r) .
- 11: Store (s, a, r, s') into $\mathcal{D}^{replay}$, overwriting oldest self-generated transition if over capacity
- 12: Sample a mini-batch of n transitions from $\mathcal{D}^{replay}$ with prioritization
- 13: Calculate loss $J(Q)$ using target network
- 14: Perform a gradient descent step to update θ
- 15: **if** $t \bmod \tau = 0$ **then** $\theta' \leftarrow \theta$ **end if**
- 16: $s \leftarrow s'$
- 17: **end for**

实验装置

我们在 Arcade 学习环境 (ALE) 上评估了 DQfD (Belle et al., 2013 年)。ALE 是一组 Atari 游戏, 它是 DQN 的标准基准, 并且包含许多人类仍然比最好的学习代理表现更好的游戏。代理从游戏屏幕的下采样 84×84 图像 (已转换为灰度) 玩 Atari 游戏, 并且代理将其四个帧堆叠在一起作为其状态。代理必须为每场比赛输出 18 个可能的动作之一。该代理应用 0.99 的折扣因子, 并且对四个 Atari 框架重复其所有操作。每个情节都通过最多 30 个无操作操作进行初始化, 以提供随机起始位置。报告的分数是 Atari 游戏中的分数, 无论智能体内部如何表示奖励。

对于我们所有的实验, 我们评估了三种不同的算法, 每种算法在四次试验中进行平均:

- 带人工演示的完整 DQfD 算法
- PDD DQN 学习, 无需任何演示数据
- 根据演示数据进行监督模仿, 无需任何环境交互

我们对六款 Atari 游戏的所有算法进行了非正式的参数调整, 然后对整组游戏使用相同的参数。算法使用的参数显示在附录中。我们对优先级和 n 步返回参数的粗略搜索得出了 DQfD 和 PDD DQN 相同的最佳参数。PDD DQN

differs from DQfD because it does not have demonstration data, pre-training, supervised losses, or regularization losses. We included n-step returns in PDD DQN to provide a better baseline for comparison between DQfD and PDD DQN. All three algorithms use the dueling state-advantage convolutional network architecture (Wang et al. 2016).

For the supervised imitation comparison, we performed supervised classification of the demonstrator’s actions using a cross-entropy loss, with the same network architecture and L2 regularization used by DQfD. The imitation algorithm did not use any TD loss. Imitation learning only learns from the pre-training and not from any additional interactions.

We ran experiments on a randomly selected subset of 42 Atari games. We had a human player play each game between three and twelve times. Each episode was played either until the game terminated or for 20 minutes. During game play, we logged the agent’s state, actions, rewards, and terminations. The human demonstrations range from 5,574 to 75,472 transitions per game. DQfD learns from a very small dataset compared to other similar work, as AlphaGo (Silver et al. 2016) learns from 30 million human transitions, and DQN (Mnih et al. 2015) learns from over 200 million frames. DQfD’s smaller demonstration dataset makes it more difficult to learn a good representation without over-fitting. The demonstration scores for each game are shown in a table in the Appendix. Our human demonstrator is much better than PDD DQN on some games (e.g. Private Eye, Pitfall), but much worse than PDD DQN on many games (e.g. Breakout, Pong).

We found that in many of the games where the human player is better than DQN, it was due to DQN being trained with all rewards clipped to 1. For example, in Private Eye, DQN has no reason to select actions that reward 25,000 versus actions that reward 10. To make the reward function used by the human demonstrator and the agent more consistent, we used unclipped rewards and converted the rewards using a log scale: $r_{agent} = \text{sign}(r) \cdot \log(1 + |r|)$. This transformation keeps the rewards over a reasonable scale for the neural network to learn, while conveying important information about the relative scale of individual rewards. These adapted rewards are used internally by all the algorithms in our experiments. Results are still reported using actual game scores as is typically done in the Atari literature (Mnih et al. 2015).

Results

First, we show learning curves in Figure 1 for three games: Hero, Pitfall, and Road Runner. On Hero and Pitfall, the human demonstrations enable DQfD to achieve a score higher than any previously published result. Videos for both games are available at <https://www.youtube.com/watch?v=JR6wmLaYuu4>. On Hero, DQfD achieves a higher score than any of the human demonstrations as well as any previously published result. Pitfall may be the most difficult Atari game, as it has very sparse positive rewards and dense negative rewards. No previous approach achieved any positive rewards on this game, while DQfD’s best score on this game averaged over a 3 million step period is 394.0.

On Road Runner, agents typically learn super-human policies with a score exploit that differs greatly from human play. Our demonstrations are only human and have a maximum score of 20,200. Road Runner is the game with the smallest set of human demonstrations (only 5,574 transitions). Despite these factors, DQfD still achieves a higher score than PDD DQN for the first 36 million steps and matches PDD DQN’s performance after that.

The right subplot in Figure 1 shows the ratio of how often the demonstration data was sampled versus how much it would be sampled with uniform sampling. For the most difficult games like Pitfall and Montezuma’s Revenge, the demonstration data is sampled more frequently over time. For most other games, the ratio converges to a near constant level, which differs for each game.

In real world tasks, the agent must perform well from its very first action and must learn quickly. DQfD performed better than PDD DQN on the first million steps on 41 of 42 games. In addition, on 31 games, DQfD starts out with higher performance than pure imitation learning, as the addition of the TD loss helps the agent generalize the demonstration data better. On average, PDD DQN does not surpass the performance of DQfD until 83 million steps into the task and never surpasses it in mean scores.

In addition to boosting initial performance, DQfD is able to leverage the human demonstrations to learn better policies on the most difficult Atari games. We compared DQfD’s scores over 200 million steps with that of other deep reinforcement learning approaches: DQN, Double DQN, Prioritized DQN, Dueling DQN, PopArt, DQN+CTS, and DQN+PixelCNN (Mnih et al. 2015; van Hasselt, Guez, and Silver 2016; Schaul et al. 2016; Wang et al. 2016; van Hasselt et al. 2016; Ostrovski et al. 2017). We took the best 3 million step window averaged over 4 seeds for the DQfD scores. DQfD achieves better scores than these algorithms on 11 of 42 games, shown in Table 1. Note that we do not compare with A3C (Mnih et al. 2016) or Reactor (Gruslys et al. 2017) as the only published results are for human starts, and we do not compare with UNREAL (Jaderberg et al. 2016) as they select the best hyper-parameters per game. Despite this fact, DQfD still out-performs the best UNREAL results on 10 games. DQN with count-based exploration (Ostrovski et al. 2017) is designed for and achieves the best results on the most difficult exploration games. On the six sparse reward, hard exploration games both algorithms were run on, DQfD learns better policies on four of six games.

DQfD out-performs the worst demonstration episode it was given on in 29 of 42 games and it learns to play better than the best demonstration episode in 14 of the games: Amidar, Atlantis, Boxing, Breakout, Crazy Climber, Defender, Enduro, Fishing Derby, Hero, James Bond, Kung Fu Master, Pong, Road Runner, and Up N Down. In comparison, pure imitation learning is worse than the demonstrator’s performance in every game.

Figure 2 shows comparisons of DQfD with λ_1 and λ_2 set to 0, on two games where DQfD achieved state-of-the-art results: Montezuma’s Revenge and Q-Bert. As expected,

与 DQfD 不同，因为它没有演示数据、预训练、监督损失或正则化损失。我们在 PDD DQN 中包含了 n 步回报，以便为 DQfD 和 PDD DQN 之间的比较提供更好的基线。所有三种算法都使用决斗状态优势卷积网络架构（Wang et al. 2016）。

对于监督模仿比较，我们使用交叉熵损失对演示者的行为进行监督分类，并使用与 DQfD 相同的网络架构和 L2 正则化。仿算法没有使用任何 TD 损失。模仿学习仅从预训练中学习，而不是从任何额外的交互中学习。

我们对随机选择的 42 款 Atari 游戏子集进行了实验。我们让一名人类玩家玩每场游戏三到十二次。每集要么玩到游戏结束，要么玩 20 分钟。在游戏过程中，我们记录了代理的状态、操作、奖励和终止。每场比赛的人类演示范围为 5,574 到 75,472 次转换。与其他类似工作相比，DQfD 从非常小的数据集进行学习，如 AlphaGo（Silver 等人，2016）从 3000 万个人类转换中学习，而 DQN（Mnih 等人，2015）从超过 2 亿帧学习。DQfD 较小的演示数据集使得在不过度拟合的情况下学习良好的表示更加困难。每场比赛的示范得分如附录中的表格所示。我们的人类演示者在某些游戏（例如 Private Eye、Pitfall）上比 PDD DQN 好得多，但在许多游戏（例如 Breakout、Pong）上比 PDD DQN 差得多。

我们发现，在许多人类玩家比 DQN 更好的游戏中，这是由于 DQN 是在所有奖励被削减为 1 的情况下进行训练的。例如，在 Private Eye 中，DQN 没有理由选择奖励 25,000 的动作而不是奖励 10 的动作。为了使人类演示者和代理使用的奖励函数更加一致，我们使用未削减的奖励并使用对数尺度转换奖励：

$r_{agent} = \text{sign}(r) \cdot \log(1 + |r|)$ 。这种转换使奖励保持在一个合理的范围内，以便神经网络学习，同时传达有关个体奖励相对规模的重要信息。这些适应的奖励由我们实验中的所有算法内部使用。结果仍然使用实际游戏分数来报告，就像 Atari 文献中通常所做的那样（Mnih 等人，2015 年）。

结果

首先，我们在图 1 中显示了三个游戏的学习曲线：Hero、Pitfall 和 Road Runner。在 Hero 和 Pitfall 中，人类演示使 DQfD 获得了比之前发布的任何结果都要高的分数。两款游戏的视频均可在 <https://www.youtube.com/watch?v=JR6wmLaYuu4> 上找到

在 Hero 上，DQfD 获得了比任何人类演示以及之前发布的结果更高的分数。Pitfall 可能是最难的 Atari 游戏，因为它的正奖励非常稀疏，而负奖励却非常密集。之前的方法都没有在这款游戏上获得任何积极的奖励，而 DQfD 在这款游戏上 300 万步期间的平均最佳得分为 39.40。

在《Road Runner》中，智能体通常会通过与人类游戏截然不同的分数利用来学习超人类策略。我们的演示仅限于人类，最高得分为 20,200。Road Runner 是人类演示最少的游戏（只有 5,574 个转换）。尽管存在这些因素，DQfD 在前 3600 万步中仍然取得了比 PDD DQN 更高的分数，并且此后与 PDD DQN 的性能相匹配。

图 1 中的右侧子图显示了演示数据的采样频率与均匀采样的采样次数之比。对于像《陷阱》和《蒙特祖玛的复仇》这样最困难的游戏，随着时间的推移，演示数据的采样会更加频繁。对于大多数其他游戏，该比率收敛到接近恒定的水平，该水平因游戏而异。

在现实世界的任务中，智能体必须从第一个动作就表现良好，并且必须快速学习。在 42 场比赛中的 41 场比赛中，DQfD 在前 100 万步中的表现优于 PDD DQN。此外，在 31 场比赛中，DQfD 开始时比纯模仿学习具有更高的性能，因为 TD 损失的添加有助于代理更好地泛化演示数据。平均而言，PDD DQN 在执行 8300 万步任务之前不会超过 DQfD 的性能，并且在平均分数上从未超过它。

除了提高初始性能外，DQfD 还能够利用人类演示来学习针对最困难的 Atari 游戏的更好策略。我们将 DQfD 超过 2 亿步的分数与其他深度强化学习方法的分数进行了比较：DQN、Double DQN、Prioritized DQN、Dueling DQN、PopArt、DQN+CTS 和 DQN+PixelCNN（Mnih 等人，2015 年；van Hasselt、Guez 和 Silver，2016 年；Schaul 等人，2016 年；Wang 等人，2016 年）。2016；奥斯特洛夫斯基等人，2017）。我们采用平均超过 4 个种子的最佳 300 万步窗口作为 DQfD 分数。DQfD 在 42 场比赛中的 11 场上取得了比这些算法更好的分数，如表 1 所示。请注意，我们不与 A3C（Mnih 等人，2016 年）或 Reactor（Gruslys 等人，2017 年）进行比较，因为唯一公布的结果是针对人类开始的，并且我们不与 UNREAL（Jaderberg 等人，2016 年）进行比较，因为它们选择每场比赛的最佳超参数。尽管如此，DQfD 在 10 场比赛中的表现仍然优于 UNREAL 的最佳成绩。具有基于计数的探索的 DQN（Ostrovski 等人，2017）专为最困难的探索游戏而设计，并在最困难的探索游戏中取得了最佳结果。在运行两种算法的六种稀疏奖励、艰苦探索游戏中，DQfD 在六种游戏中的四种中学习到了更好的策略。

DQfD 在 42 场比赛中的 29 场中表现超过了最差的演示集，并且在 14 场游戏中学习到了比最好的演示集更好的表现：Amidar、Atlantis、Boxing、Breakout、Crazy Climber、Defender、Enduro、Fishing Derby、Hero、James Bond、Kung Fu Master、Pong、Road Runner 和 Up N Down。相比之下，纯粹的模仿学习在每场比赛中都比演示者的表现更差。

图 2 显示了 λ_1 和 λ_2 设置为 0 时 DQfD 在两个游戏中的比较，其中 DQfD 取得了最先进的结果：Montezuma's Revenge 和 Q-Bert。正如预期的那样，

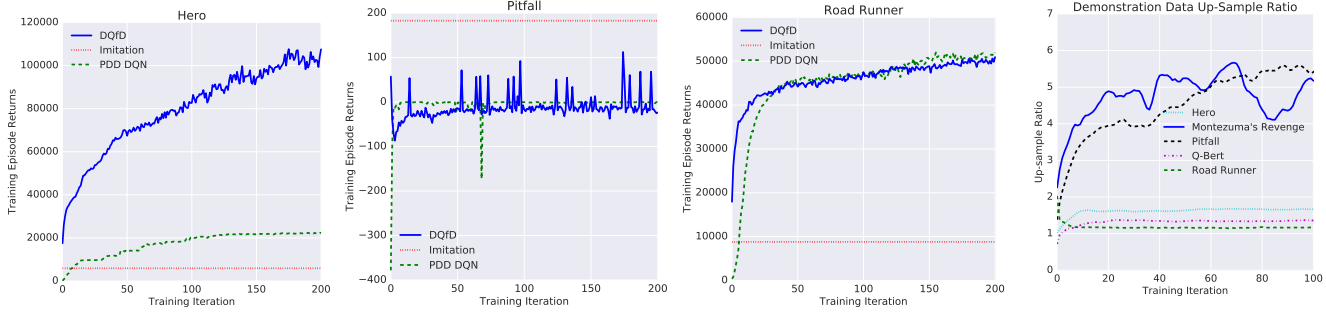


Figure 1: On-line scores of the algorithms on the games of Hero, Pitfall, and Road Runner. On Hero and Pitfall, DQfD leverages the human demonstrations to achieve a higher score than any previously published result. The last plot shows how much more frequently the demonstration data was sampled than if data were sampled uniformly, for five different games.

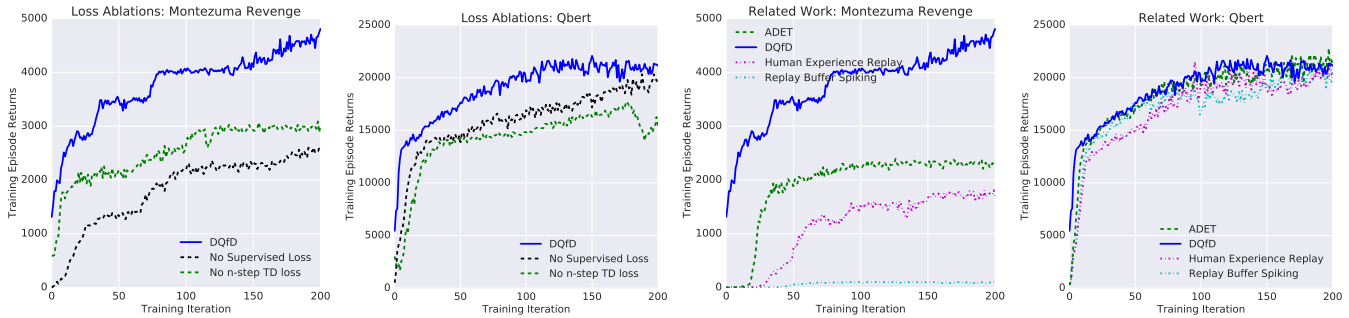


Figure 2: The left plots show on-line rewards of DQfD with some losses removed on the games of Montezuma's Revenge and Q-Bert. Removing either loss degrades the performance of the algorithm. The right plots compare DQfD with three algorithms from the related work section. The other approaches do not perform as well as DQfD, particularly on Montezuma's Revenge.

Game	DQfD	Prev. Best	Algorithm
Alien	4745.9	4461.4	Dueling DQN (Wang et al. 2016)
Asteroids	3796.4	2869.3	PopArt (van Hasselt et al. 2016)
Atlantis	920213.9	395762.0	Prior. Dueling DQN (Wang et al. 2016)
Battle Zone	41971.7	37150.0	Dueling DQN (Wang et al. 2016)
Gravitar	1693.2	859.1	DQN+PixelCNN (Ostrovski et al. 2017)
Hero	105929.4	23037.7	Prioritized DQN (Schaul et al. 2016)
Montezuma Revenge	4739.6	3705.5	DQN+CTS (Ostrovski et al. 2017)
Pitfall	50.8	0.0	Prior. Dueling DQN (Wang et al. 2016)
Private Eye	40908.2	15806.5	DQN+PixelCNN (Ostrovski et al. 2017)
Q-Bert	21792.7	19220.3	Dueling DQN (Wang et al. 2016)
Up N Down	82555.0	44939.6	Dueling DQN (Wang et al. 2016)

Table 1: Scores for the 11 games where DQfD achieves higher scores than any previously published deep RL result using random no-op starts. Previous results take the best agent at its best iteration and evaluate it for 100 episodes. DQfD scores are the best 3 million step window averaged over four seeds, which is 508 episodes on average.

pre-training without any supervised loss results in a network trained towards ungrounded Q-learning targets and the agent starts with much lower performance and is slower to improve. Removing the n-step TD loss has nearly as large an impact on initial performance, as the n-step TD loss greatly helps in learning from the limited demonstration dataset.

The right subplots in Figure 2 compare DQfD with three related algorithms for leveraging demonstration data in DQN:

- Replay Buffer Spiking (RBS) (Lipton et al. 2016)
- Human Experience Replay (HER) (Hosu and Rebedea 2016)
- Accelerated DQN with Expert Trajectories (ADET) (Lakshminarayanan, Ozair, and Bengio 2016)

RBS is simply PDD DQN with the replay buffer initially full of demonstration data. HER keeps the demonstration data and mixes demonstration and agent data in each mini-batch. ADET is essentially DQfD with the large margin supervised loss replaced with a cross-entropy loss. The results show that all three of these approaches are worse than DQfD in both games. Having a supervised loss is critical to good performance, as both DQfD and ADET perform much better than the other two algorithms. All the algorithms use the exact same demonstration data used for DQfD. We included the prioritized replay mechanism and the n-step returns in all of these algorithms to make them as strong a comparison as possible.

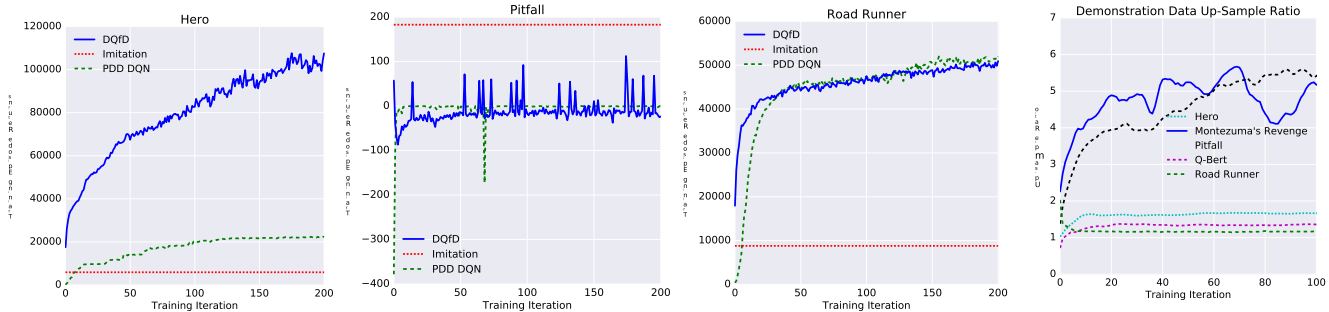


图1: 《Hero》、《Pitfall》和《Road Runner》游戏的算法在线得分。在 Hero 和 Pitfall 中, DQfD 利用人类演示获得了比之前发布的任何结果更高的分数。最后一张图显示了对于五种不同的游戏, 演示数据的采样频率比统一采样数据的频率要高得多。

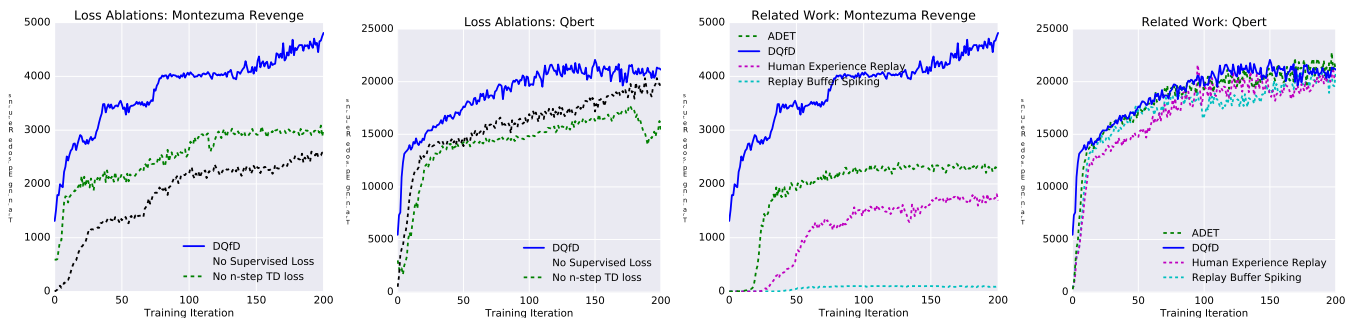


图 2: 左图显示了 DQfD 的在线奖励, 并消除了 Montezuma's Revenge 和 Q-Bert 游戏中的一些损失。消除任何一个损失都会降低算法的性能。右图将 DQfD 与相关工作部分中的三种算法进行比较。其他方法的表现不如 DQfD, 特别是在《蒙特祖玛的复仇》中。

没有任何监督损失的预训练会导致网络针对无基础的 Q 学习目标进行训练, 并且代理一开始的性能要低得多, 并且改进速度更慢。消除 n 步 TD 损失对初始性能的影响几乎同样大, 因为 n 步 TD 损失极大地有助于从有限的演示数据集中学习。

图 2 中的右侧子图将 DQfD 与利用 DQN 中演示数据的三种相关算法进行了比较:

- 重播缓冲区尖峰 (RBS) (Lipton 等人, 2016)
- 人类经验回放 (HER) (Hosu 和 Rebedea 2016)
- 使用专家轨迹 (ADET) 加速 DQN (Lakshminarayanan、Ozair 和 Bengio 2016)

RBS 只是 PDD DQN, 重播缓冲区最初充满了演示数据。HER 保留演示数据并在每个小批量中混合演示数据和代理数据。ADET 本质上是 DQfD, 用交叉熵损失代替了大裕度监督损失。结果表明, 这三种方法在两款游戏中都比 DQfD 差。受监督的损失对于良好的性能至关重要, 因为 DQfD 和 ADET 的性能都比其他两种算法好得多。所有算法都使用与 DQfD 完全相同的演示数据。我们在所有这些算法中都包含了优先重放机制和 n 步返回, 以使它们具有尽可能强的比较性。

Game	DQfD	Prev. Best	Algorithm
Alien	4745.9	4461.4	Dueling DQN (Wang et al. 2016)
Asteroids	3796.4	2869.3	PopArt (van Hasselt et al. 2016)
Atlantis	920213.9	395762.0	Prior. Dueling DQN (Wang et al. 2016)
Battle Zone	41971.7	37150.0	Dueling DQN (Wang et al. 2016)
Gravitar	1693.2	859.1	DQN+PixelCNN (Ostrovski et al. 2017)
Hero	105929.4	23037.7	Prioritized DQN (Schaul et al. 2016)
Montezuma Revenge	4739.6	3705.5	DQN+CTS (Ostrovski et al. 2017)
Pitfall	50.8	0.0	Prior. Dueling DQN (Wang et al. 2016)
Private Eye	40908.2	15806.5	DQN+PixelCNN (Ostrovski et al. 2017)
Q-Bert	21792.7	19220.3	Dueling DQN (Wang et al. 2016)
Up N Down	82555.0	44939.6	Dueling DQN (Wang et al. 2016)

表 1: DQfD 的 11 场比赛得分高于之前发布的任何使用随机无操作启动的深度 RL 结果。之前的结果采用最佳迭代的最佳智能体并对其进行 100 集评估。DQfD 分数是四个种子的平均最佳 300 万步窗口, 平均为 508 集。

Discussion

The learning framework that we have presented in this paper is one that is very common in real world problems such as controlling data centers, autonomous vehicles (Hester and Stone 2013), or recommendation systems (Shani, Heckerman, and Brafman 2005). In these problems, typically there is no accurate simulator available, and learning must be performed on the real system with real consequences. However, there is often data available of the system being operated by a previous controller. We have presented a new algorithm called DQfD that takes advantage of this data to accelerate learning on the real system. It first pre-trains solely on demonstration data, using a combination of 1-step TD, n-step TD, supervised, and regularization losses so that it has a reasonable policy that is a good starting point for learning in the task. Once it starts interacting with the task, it continues learning by sampling from both its self-generated data as well as the demonstration data. The ratio of both types of data in each mini-batch is automatically controlled by a prioritized-replay mechanism.

We have shown that DQfD gets a large boost in initial performance compared to PDD DQN. DQfD has better performance on the first million steps than PDD DQN on 41 of 42 Atari games, and on average it takes DQN 82 million steps to match DQfD’s performance. On most real world tasks, an agent may never get hundreds of millions of steps from which to learn. We also showed that DQfD out-performs three other algorithms for leveraging demonstration data in RL. The fact that DQfD out-performs all these algorithms makes it clear that it is the better choice for any real-world application of RL where this type of demonstration data is available.

In addition to its early performance boost, DQfD is able to leverage the human demonstrations to achieve state-of-the-art results on 11 Atari games. Many of these games are the hardest exploration games (i.e. Montezuma’s Revenge, Pitfall, Private Eye) where the demonstration data can be used in place of smarter exploration. This result enables the deployment of RL to problems where more intelligent exploration would otherwise be required.

DQfD achieves these results despite having a very small amount of demonstration data (5,574 to 75,472 transitions per game) that can be easily generated in just a few minutes of gameplay. DQN and DQfD receive three orders of magnitude more interaction data for RL than demonstration data. DQfD demonstrates the gains that can be achieved by adding just a small amount of demonstration data with the right algorithm. As the related work comparison shows, naively adding (e.g. only pre-training or filling the replay buffer) this small amount of data to a pure deep RL algorithm does not provide similar benefit and can sometimes be detrimental.

These results may seem obvious given that DQfD has access to privileged data, but the rewards and demonstrations are mathematically dissimilar training signals, and naive approaches to combining them can have disastrous results. Simply doing supervised learning on the human demonstrations is not successful, while DQfD learns to out-perform the best demonstration in 14 of 42 games. DQfD also out-performs three prior algorithms for incorporating demon-

stration data into DQN. We argue that the combination of all four losses during pre-training is critical for the agent to learn a coherent representation that is not destroyed by the switch in training signals after pre-training. Even after pre-training, the agent must continue using the expert data. In particular, the right sub-figure of Figure 1 shows that the ratio of expert data needed (selected by prioritized replay) grows during the interaction phase for the most difficult exploration games, where the demonstration data becomes more useful as the agent reaches new screens in the game. RBS shows an example where just having the demonstration data initially is not enough to provide good performance.

Learning from human demonstrations is particularly difficult. In most games, imitation learning is unable to perfectly classify the demonstrator’s actions even on the demonstration dataset. Humans may play the games in a way that differs greatly from a policy that an agent would learn, and may be using information that is not available in the agent’s state representation. In future work, we plan to measure these differences between demonstration and agent data to inform approaches that derive more value from the demonstrations. Another future direction is to apply these concepts to domains with continuous actions, where the classification loss becomes a regression loss.

Acknowledgments

The authors would like to thank Keith Anderson, Chris Apps, Ben Coppin, Joe Fenton, Nando de Freitas, Chris Gamble, Thore Graepel, Georg Ostrovski, Cosmin Paduraru, Jack Rae, Amir Sadik, Jon Scholz, David Silver, Toby Pohlen, Tom Stepleton, Ziyu Wang, and many others at DeepMind for insightful discussions, code contributions, and other efforts.

References

- [Abbeel et al. 2007] Abbeel, P.; Coates, A.; Quigley, M.; and Ng, A. Y. 2007. An application of reinforcement learning to aerobatic helicopter flight. In *Advances in Neural Information Processing Systems (NIPS)*.
- [Bellemare et al. 2013] Bellemare, M. G.; Naddaf, Y.; Veness, J.; and Bowling, M. 2013. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research (JAIR)* 47:253–279.
- [Brys et al. 2015] Brys, T.; Harutyunyan, A.; Suay, H.; Chernova, S.; Taylor, M.; and Nowé, A. 2015. Reinforcement learning from demonstration through shaping. In *International Joint Conference on Artificial Intelligence (IJCAI)*.
- [Cederborg et al. 2015] Cederborg, T.; Grover, I.; Isbell, C.; and Thomaz, A. 2015. Policy shaping with human teachers. In *International Joint Conference on Artificial Intelligence (IJCAI 2015)*.
- [Chemali and Lezaric 2015] Chemali, J., and Lezaric, A. 2015. Direct policy iteration from demonstrations. In *International Joint Conference on Artificial Intelligence (IJCAI)*.
- [Duan et al. 2017] Duan, Y.; Andrychowicz, M.; Stadie, B. C.; Ho, J.; Schneider, J.; Sutskever, I.; Abbeel, P.; and

讨论

我们在本文中提出的学习框架是现实世界问题中非常常见的框架，例如控制数据中心、自动驾驶车辆（Hester 和 Stone 2013）或推荐系统（Shani、Heckerman 和 Braffman 2005）。在这些问题中，通常没有可用的精确模拟器，并且必须在真实系统上进行学习并产生真实的结果。然而，通常有由先前控制器操作的系统的可用数据。我们提出了一种名为 DQfD 的新算法，它利用这些数据来加速真实系统的学习。它首先仅在演示数据上进行预训练，结合使用 1 步 TD、 n 步 TD、监督和正则化损失，以便它具有合理的策略，成为任务学习的良好起点。一旦它开始与任务交互，它就会通过从自身生成的数据和演示数据中采样来继续学习。每个小批量中两种类型数据的比例由优先重播机制自动控制。

我们已经证明，与 PDD DQN 相比，DQfD 的初始性能得到了大幅提升。在 42 款 Atari 游戏中的 41 款中，DQfD 在前 100 万步上的性能优于 PDD DQN，并且平均需要 DQN 8200 万步才能与 DQfD 的性能相匹配。在大多数现实世界的任务中，智能体可能永远不会获得数亿个步骤来学习。我们还表明，在利用 RL 中的演示数据方面，DQfD 的性能优于其他三种算法。DQfD 优于所有这些算法的事实清楚地表明，对于任何可获得此类演示数据的 RL 实际应用来说，它是更好的选择。

除了早期性能提升之外，DQfD 还能够利用人类演示在 11 款 Atari 游戏上取得最先进的结果。其中许多游戏都是最难的探索游戏（例如《蒙特祖玛的复仇》、《陷阱》、《私家侦探》），可以使用演示数据来代替更智能的探索。这一结果使得强化学习能够部署到需要更智能探索的问题上。

尽管 DQfD 具有非常少量的演示数据（每场游戏 5,574 到 75,472 个转换），但可以在短短几分钟的游戏过程中轻松生成，但仍实现了这些结果。DQN 和 DQfD 接收到的 RL 交互数据比演示数据多三个数量级。DQfD 演示了通过使用正确的算法添加少量演示数据即可实现的收益。正如相关工作比较所示，天真地向纯深度 RL 算法添加（例如仅预训练或填充重放缓冲区）少量数据并不能提供类似的好处，有时甚至可能是有害的。

鉴于 DQfD 可以访问特权数据，这些结果似乎是显而易见的，但奖励和演示在数学上是不同的训练信号，将它们结合起来的幼稚方法可能会产生灾难性的结果。简单地对人类演示进行监督学习是不成功的，而 DQfD 在 42 场比赛中的 14 场中表现优于最佳演示。DQfD 还优于三种先前的算法，用于合并恶魔

将数据写入 DQN。我们认为，预训练期间所有四种损失的组合对于代理学习连贯的表示至关重要，该表示不会被预训练后训练信号的切换所破坏。即使在预训练之后，代理也必须继续使用专家数据。特别是，图 1 的右侧子图显示，在最困难的探索游戏的交互阶段，所需专家数据的比例（通过优先重放选择）会增长，其中随着代理到达游戏中的新屏幕，演示数据变得更加有用。RBS 展示了一个示例，其中最初仅拥有演示数据不足以提供良好的性能。

从人类的示范中学习尤其困难。在大多数游戏中，即使在演示数据集上，模仿学习也无法完美地对演示者的行为进行分类。人类玩游戏的方式可能与智能体学习的策略有很大不同，并且可能使用智能体状态表示中不可用的信息。在未来的工作中，我们计划测量演示数据和代理数据之间的差异，以便为从演示中获得更多价值的方法提供信息。另一个未来的方向是将这些概念应用到具有连续动作的领域，其中分类损失变成回归损失。

致谢

作者要感谢 Keith Anderson、Chris Apps、Ben Coppin、Joe Fenton、Nando de Freitas、Chris Gamble、Thore Graepel、Georg Ostrovski、Cosmin Paduraru、Jack Rae、Amir Sadik、Jon Scholz、David Silver、Toby Pohlen、Tom Stepleton、Ziyu Wang 以及 DeepMind 的许多其他人的富有洞察力的讨论、代码贡献和其他努力。

参考

[阿贝尔等人。2007]阿贝尔，P.；科茨，A.；奎格利，M.；和 Ng, A. Y. 2007。强化学习在特技直升机飞行中的应用。在 *Advances in Neural Information Processing Systems (NIPS)* 中。[贝尔马尔等人。2013]贝尔马尔，M.G.；纳达夫，Y.；Veness, J.；和 Bowling, M. 2013。街机学习环境：一般代理的评估平台。 *Journal of Artificial Intelligence Research (JAIR)* 47:253-279。[布莱斯等人。2015]布莱斯，T.；哈鲁图尼扬，A.；苏伊，H.；Chernova, S.；泰勒，M.；以及 Nowé, A. 2015。从演示到塑造的强化学习。在 *International Joint Conference on Artificial Intelligence (IJCAI)* 中。[塞德堡等人。2015]塞德堡，T.；格罗弗，I.；伊斯贝尔，C.；和 Thomaz, A. 2015。与人类教师一起制定政策。在 *International Joint Conference on Artificial Intelligence (IJCAI 2015)* 中。[Chemali 和 Lezaric 2015] Chemali, J. 和 Lezaric, A. 2015。来自演示的直接政策迭代。在 *International Joint Conference on Artificial Intelligence (IJCAI)* 中。[段等人。2017]段 Y.；安德里霍维奇，M.；斯塔迪，不列颠哥伦比亚省；何，J.；施奈德，J.；苏茨克维尔，I.；阿贝尔，P.；和

- Zaremba, W. 2017. One-shot imitation learning. *CoRR* abs/1703.07326.
- [Finn, Levine, and Abbeel 2016] Finn, C.; Levine, S.; and Abbeel, P. 2016. Guided cost learning: Deep inverse optimal control via policy optimization. In *International Conference on Machine Learning (ICML)*.
- [Gruslys et al. 2017] Gruslys, A.; Gheshlaghi Azar, M.; Bellemare, M. G.; and Munos, R. 2017. The Reactor: A Sample-Efficient Actor-Critic Architecture. *ArXiv e-prints*.
- [Hester and Stone 2013] Hester, T., and Stone, P. 2013. TEXPLORE: Real-time sample-efficient reinforcement learning for robots. *Machine Learning* 90(3).
- [Ho and Ermon 2016] Ho, J., and Ermon, S. 2016. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems (NIPS)*.
- [Hosu and Rebedea 2016] Hosu, I.-A., and Rebedea, T. 2016. Playing atari games with deep reinforcement learning and human checkpoint replay. In *ECAI Workshop on Evaluating General Purpose AI*.
- [Jaderberg et al. 2016] Jaderberg, M.; Mnih, V.; Czarnecki, W. M.; Schaul, T.; Leibo, J. Z.; Silver, D.; and Kavukcuoglu, K. 2016. Reinforcement learning with unsupervised auxiliary tasks. *CoRR* abs/1611.05397.
- [Kim et al. 2013] Kim, B.; Farahmand, A.; Pineau, J.; and Precup, D. 2013. Learning from limited demonstrations. In *Advances in Neural Information Processing Systems (NIPS)*.
- [Lakshminarayanan, Ozair, and Bengio 2016] Lakshminarayanan, A. S.; Ozair, S.; and Bengio, Y. 2016. Reinforcement learning with few expert demonstrations. In *NIPS Workshop on Deep Learning for Action and Interaction*.
- [LeCun, Bengio, and Hinton 2015] LeCun, Y.; Bengio, Y.; and Hinton, G. 2015. Deep learning. *Nature* 521(7553):436–444.
- [Levine et al. 2016] Levine, S.; Finn, C.; Darrell, T.; and Abbeel, P. 2016. End-to-end training of deep visuomotor policies. *Journal of Machine Learning (JMLR)* 17:1–40.
- [Lipton et al. 2016] Lipton, Z. C.; Gao, J.; Li, L.; Li, X.; Ahmed, F.; and Deng, L. 2016. Efficient exploration for dialog policy learning with deep BBQ network & replay buffer spiking. *CoRR* abs/1608.05081.
- [Mnih et al. 2015] Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; Petersen, S.; Beattie, C.; Sadik, A.; Antonoglou, I.; King, H.; Kumaran, D.; Wierstra, D.; Legg, S.; and Hassabis, D. 2015. Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533.
- [Mnih et al. 2016] Mnih, V.; Badia, A. P.; Mirza, M.; Graves, A.; Lillicrap, T.; Harley, T.; Silver, D.; and Kavukcuoglu, K. 2016. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning*, 1928–1937.
- [Ostrovski et al. 2017] Ostrovski, G.; Bellemare, M. G.; van den Oord, A.; and Munos, R. 2017. Count-based exploration with neural density models. *CoRR* abs/1703.01310.
- [Piot, Geist, and Pietquin 2014a] Piot, B.; Geist, M.; and Pietquin, O. 2014a. Boosted bellman residual minimization handling expert demonstrations. In *European Conference on Machine Learning (ECML)*.
- [Piot, Geist, and Pietquin 2014b] Piot, B.; Geist, M.; and Pietquin, O. 2014b. Boosted and Reward-regularized Classification for Apprenticeship Learning. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
- [Ross, Gordon, and Bagnell 2011] Ross, S.; Gordon, G. J.; and Bagnell, J. A. 2011. A reduction of imitation learning and structured prediction to no-regret online learning. In *International Conference on Artificial Intelligence and Statistics (AISTATS)*.
- [Schaal 1996] Schaal, S. 1996. Learning from demonstration. In *Advances in Neural Information Processing Systems (NIPS)*.
- [Schaul et al. 2016] Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized experience replay. In *Proceedings of the International Conference on Learning Representations*, volume abs/1511.05952.
- [Shani, Heckerman, and Brafman 2005] Shani, G.; Heckerman, D.; and Brafman, R. I. 2005. An mdp-based recommender system. *Journal of Machine Learning Research* 6:1265–1295.
- [Silver et al. 2016] Silver, D.; Huang, A.; Maddison, C. J.; Guez, A.; Sifre, L.; van den Driessche, G.; Schrittwieser, J.; Antonoglou, I.; Panneershelvam, V.; Lanctot, M.; Dieleman, S.; Grewe, D.; Nham, J.; Kalchbrenner, N.; Sutskever, I.; Lillicrap, T.; Leach, M.; Kavukcuoglu, K.; Graepel, T.; and Hassabis, D. 2016. Mastering the game of Go with deep neural networks and tree search. *Nature* 529:484–489.
- [Suay et al. 2016] Suay, H. B.; Brys, T.; Taylor, M. E.; and Chernova, S. 2016. Learning from demonstration for shaping through inverse reinforcement learning. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
- [Subramanian, Jr., and Thomaz 2016] Subramanian, K.; Jr., C. L. I.; and Thomaz, A. 2016. Exploration from demonstration for interactive reinforcement learning. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
- [Sun et al. 2017] Sun, W.; Venkatraman, A.; Gordon, G. J.; Boots, B.; and Bagnell, J. A. 2017. Deeply aggregated: Differentiable imitation learning for sequential prediction. *CoRR* abs/1703.01030.
- [Sutton and Barto 1998] Sutton, R. S., and Barto, A. G. 1998. *Introduction to reinforcement learning*. MIT Press.
- [Syed and Schapire 2007] Syed, U., and Schapire, R. E. 2007. A game-theoretic approach to apprenticeship learning. In *Advances in Neural Information Processing Systems (NIPS)*.
- [Syed, Bowling, and Schapire 2008] Syed, U.; Bowling, M.; and Schapire, R. E. 2008. Apprenticeship learning using linear programming. In *International Conference on Machine Learning (ICML)*.

Zaremba, W. 2017. 一次性模仿学习。 *CoRR* 绝对/1703.07326。 [Finn、Levine 和 Abbeel 2016] Finn, C.; 莱文, S.; 和 Abbeel, P. 2016. 引导成本学习: 通过策略优化进行深度逆最优控制。在 *International Conference on Machine Learning (ICML)* 中。 [Gruslys 等人。 2017] Gruslys, A.; 格什拉吉·阿扎尔, M.; 贝尔马尔, M.G.; 和 Munos, R. 2017. Reactor: 一个样本高效的 Actor-Critic 架构。 *ArXiv e-prints*。 [Hester 和 Stone 2013] Hester, T. 和 Stone, P. 2013. TEXPLORE: 机器人的实时样本高效强化学习。 *Machine Learning* 90(3)。 [Ho and Ermon 2016] Ho, J., and Ermon, S. 2016. 生成对抗性模仿学习。在 *Advances in Neural Information Processing Systems (NIPS)* 中。 [Hosu 和 Rebedea 2016] Hosu, I.-A. 和 Rebedea, T. 2016. 通过深度强化学习和人工检查点重放玩 Atari 游戏。在 *ECAI Workshop on Evaluating General Purpose AI* 中。 [贾德伯格等人。 2016] 贾德伯格, M.; 姆尼赫, V.; Czarniecki, W.M.; 斯考尔, T.; 雷波, J.Z.; 银, D.; 和 Kavukcuoglu, K. 2016. 使用无监督辅助任务的强化学习。 *CoRR* 绝对/1611.05397。 [金等人。 2013] 金, B.; 法拉曼德, A.; 皮诺, J.; 和 Precup, D. 2013. 从有限的演示中学习。在 *Advances in Neural Information Processing Systems (NIPS)* 中。 [Lakshminarayanan, Ozair 和 Bengio 2016] Lakshminarayanan, A.S.; 奥扎尔, S.; 和 Bengio, Y. 2016. 几乎没有专家演示的强化学习。在 *NIPS Workshop on Deep Learning for Action and Interaction* 中。 [LeCun、Bengio 和 Hinton 2015] LeCun, Y.; 本吉奥, Y.; 和 Hinton, G. 2015. 深度学习。 *Nature* 521 (7553): 436-444。 [莱文等人。 2016] 莱文, S.; 芬恩, C.; 达雷尔, T.; 和 Abbeel, P. 2016. 深度视觉运动策略的端到端培训。 *Journal of Machine Learning (JMLR)* 17:1-40。 [利普顿等人。 2016] 利普顿, Z.C.; 高, J. 李, L. 李, X. 艾哈迈德, F.; 和 Deng, L. 2016. 通过深度 BBQ 网络和重放缓冲区尖峰进行对话策略学习的有效探索。 *CoRR* 绝对/1608.05081。 [Mnih 等人。 2015] 姆尼赫, V.; 卡武克库奥卢, K.; 银, D.; 鲁苏, A.A.; 维内斯, J.; 贝尔马尔, M.G.; 格雷夫斯, A.; 里德-米勒, M.; 菲杰兰, A.K.; 奥斯特洛夫斯基, G.; 彼得森, S.; 比蒂, C.; 萨迪克, A.; 安东诺格鲁, I.; 金, H.; 库马兰, D.; 维尔斯特拉, D.; 莱格, S.; 和 Hassabis, D. 2015. 通过深度强化学习实现人类水平的控制。 *Nature* 518 (7540): 529-533。 [Mnih 等人。 2016] 姆尼赫, V.; 巴迪亚, A.P.; 米尔扎, M.; 格雷夫斯, A.; 利利克拉普, T.; 哈雷, T.; 银, D.; 和 Kavukcuoglu, K. 2016. 深度强化学习的异步方法。在 *International Conference on Machine Learning*, 1928-1937。 [奥斯特洛夫斯基等人。 2017] 奥斯特洛夫斯基, G.; 贝尔马尔, M.G.; 范登奥尔德, A.; 和 Munos, R. 2017. 使用神经密度模型进行基于计数的探索。 *CoRR* 绝对/1703.01310。

[Piot、Geist 和 Pietquin 2014a] Piot, B.; 盖斯特, M.; 和 Pietquin, O. 2014a. 增强了贝尔曼剩余最小化处理专家演示。在 *European Conference on Machine Learning (ECML)* 中。 [Piot、Geist 和 Pietquin 2014b] Piot, B.; 盖斯特, M.; 和 Pietquin, O. 2014b. 学徒学习的强化和奖励正规化分类。在 *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)* 中。 [罗斯、戈登和巴格内尔 2011] 罗斯, S.; 戈登, G.J.; 和 Bagnell, J. A. 2011. 将模仿学习和结构化预测减少为无悔在线学习。在 *International Conference on Artificial Intelligence and Statistics (AISTATS)* 中。 [Schaal 1996] Schaal, S. 1996. 从示范中学习。在 *Advances in Neural Information Processing Systems (NIPS)* 中。 [斯考尔等人。 2016] 斯考尔, T.; 全, J.; 安东诺格鲁, I.; Silver, D. 2016. 优先体验重播。在 *Proceedings of the International Conference on Learning Representations* 中, 卷abs/1511.05952。 [Shani、Heckerman 和 Brafman 2005] Shani, G.; 赫克曼, D.; 和 Brafman, R. I. 2005. 基于 mdp 的推荐系统。 *Journal of Machine Learning Research* 6: 1265-1295。 [银等人。 2016] 银, D.; 黄, A. 麦迪逊, C.J.; 格斯, A.; 西弗雷, L.; 范登德里斯切, G.; 施里特威瑟, J.; 安东诺格鲁, I.; 潘内斯谢尔瓦姆, V.; 兰克托, M.; 迪勒曼, S.; 格鲁, D.; 纳姆, J.; 卡尔奇布伦纳, N.; 苏茨克维尔, I.; 利利克拉普, T.; 利奇, M.; 卡武克库奥卢, K.; 格雷佩尔, T.; 和 Hassabis, D. 2016. 通过深度神经网络和树搜索掌握围棋游戏。 *Nature* 529: 484-489。 [苏艾等人。 2016] Suay, H.B.; 布莱斯, T.; 泰勒, M.E.; 和 Chernova, S. 2016. 通过逆强化学习从示范中学习塑造。在 *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)* 中。 [Subramanian, Jr. 和 Thomaz 2016] Subramanian, K.; Jr., C.L.I.; 和 Thomaz, A. 2016. 交互式强化学习演示的探索。在 *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)* 中。 [孙等人。 2017] 孙 W.; 文卡特拉曼, A.; 戈登, G.J.; 布茨, B.; 和 Bagnell, J. A. 2017. 深度聚合: 用于顺序预测的可微模仿学习。 *CoRR* 绝对/1703.01030。 [Sutton 和 Barto 1998] Sutton, R. S. 和 Barto, A. G. 1998. *Introduction to reinforcement learning*。麻省理工学院出版社。 [Syed 和 Schapire 2007] Syed, U. 和 Schapire, R. E. 2007. 学徒学习的博弈论方法。在 *Advances in Neural Information Processing Systems (NIPS)* 中。 [Syed、Bowling 和 Schapire 2008] Syed, U.; 保龄球, M.; 和 Schapire, R. E. 2008. 使用线性编程的学徒学习。在 *International Conference on Machine Learning (ICML)* 中。

- [Taylor, Suay, and Chernova 2011] Taylor, M.; Suay, H.; and Chernova, S. 2011. Integrating reinforcement learning with human demonstrations of varying ability. In *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*.
- [van Hasselt et al. 2016] van Hasselt, H. P.; Guez, A.; Hessel, M.; Mnih, V.; and Silver, D. 2016. Learning values across many orders of magnitude. In *Advances in Neural Information Processing Systems (NIPS)*.
- [van Hasselt, Guez, and Silver 2016] van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double Q-learning. In *AAAI Conference on Artificial Intelligence (AAAI)*.
- [Wang et al. 2016] Wang, Z.; Schaul, T.; Hessel, M.; van Hasselt, H.; Lanctot, M.; and de Freitas, N. 2016. Dueling network architectures for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*.
- [Watter et al. 2015] Watter, M.; Springenberg, J. T.; Boedecker, J.; and Riedmiller, M. A. 2015. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing (NIPS)*.

[Taylor, Suay 和 Chernova 2011] Taylor, M.; 苏伊, H.; Chernova, S. 2011。将强化学习与不同能力的人类展示相结合。在 *International Conference on Autonomous Agents and Multiagent Systems (AAMAS)* 中。[范哈塞尔特等人。2016] 范哈塞尔特, H.P.; 格斯, A.; 赫塞尔, M.; 姆尼赫, V.; Silver, D. 2016。跨多个数量级的学习价值。在 *Advances in Neural Information Processing Systems (NIPS)* 中。[van Hasselt, Guez 和 Silver 2016] van Hasselt, H.; 格斯, A.; 和 Silver, D. 2016。深度强化学习

双Q学习。在 *AAAI Conference on Artificial Intelligence (AAAI)* 中。[王等人。2016] 王Z.; 斯考尔, T.; 赫塞尔, M.; 范哈塞尔特, H.; 兰克托, M.; 和 de Freitas, N. 2016。深度强化学习的决斗网络架构。在 *International Conference on Machine Learning (ICML)* 中。[沃特等人。2015] 沃特, M.; 斯普林伯格, J.T.; 布德克, J.; Riedmiller, M. A. 2015。嵌入控制：用于从原始图像进行控制的局部线性潜在动力学模型。在 *Advances in Neural Information Processing (NIPS)* 中。

Supplementary Material

Here are the parameters used for the three algorithms. DQfD used all of these parameters, while the other two algorithms only used the applicable parameters.

- Pre-training steps $k = 750,000$ mini-batch updates.
- N-Step Return weight $\lambda_1 = 1.0$
- Supervised loss weight $\lambda_2 = 1.0$.
- L2 regularization weight $\lambda_3 = 10^{-5}$.
- Expert margin $l(a_E, a) \text{ when } a \neq a_E = 0.8$.
- ϵ -greedy exploration with $\epsilon = 0.01$, same used by Double DQN (van Hasselt, Guez, and Silver 2016).
- Prioritized replay exponent $\alpha = 0.4$.
- Prioritized replay constants $\epsilon_a = 0.001$, $\epsilon_d = 1.0$.
- Prioritized replay importance sampling exponent $\beta_0 = 0.6$ as in (Schaul et al. 2016).
- N-step returns with $n = 10$.
- Target network update period $\tau = 10,000$ as in (Mnih et al. 2015)

补充材料

以下是三种算法使用的参数。DQfD 使用了所有这些参数，而其他两种算法仅使用了适用的参数。

- 预训练步骤 $k = 750,000$ 小批量更新。
- N步返回权重 $\lambda_1 = 1.0$
- 监督损失权重 $\lambda_2 = 1.0$ 。
- L2 正则化权重 $\lambda_3 = 10^{-5}$ 。
- 专家保证金 $l(a_E, a) \text{ when } a \neq a_E = 0.8$ 。
- ϵ -使用 $\epsilon = 0.01$ 进行贪婪探索，与 Double DQN 所使用的相同 (van Hasselt、Guez 和 Silver 2016)。
- 优先重播指数 $\alpha = 0.4$ 。
- 优先重放常量 $\epsilon_a = 0.001$ 、 $\epsilon_d = 1.0$ 。
- 优先重播重要性采样指数 $\beta_0 = 0.6$ 如 (Schaul 等人, 2016)。
- N 步返回 $n = 10$ 。
- 目标网络更新周期 $\tau = 10,000$ 如 (Mnih 等人, 2015 年)

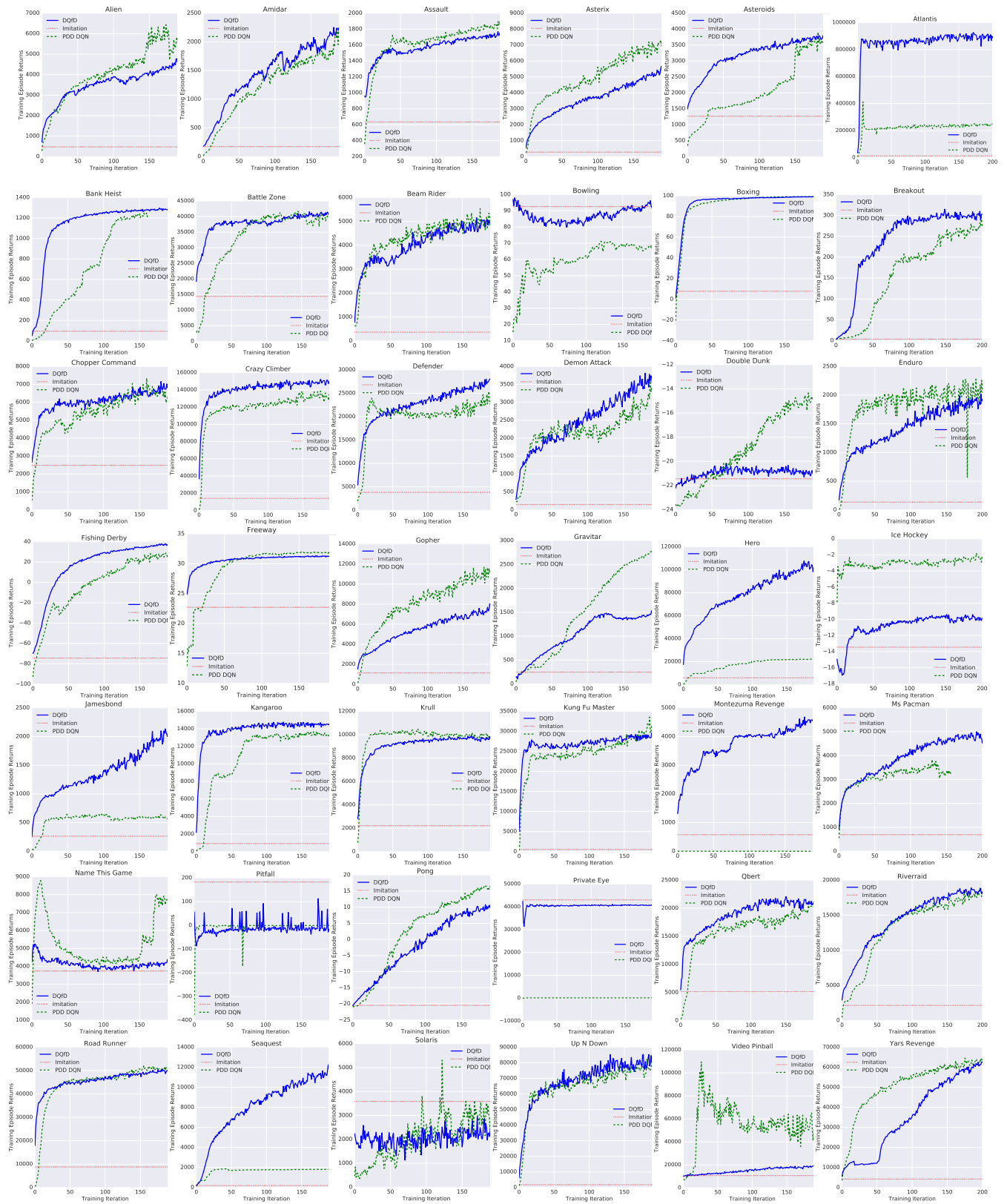


Figure 3: On-line rewards of the three algorithms on the 42 Atari games, averaged over 4 trials. Each episode is started with up to 30 random no-op actions. Scores are from the Atari game, regardless of the internal representation of reward used by the agent.

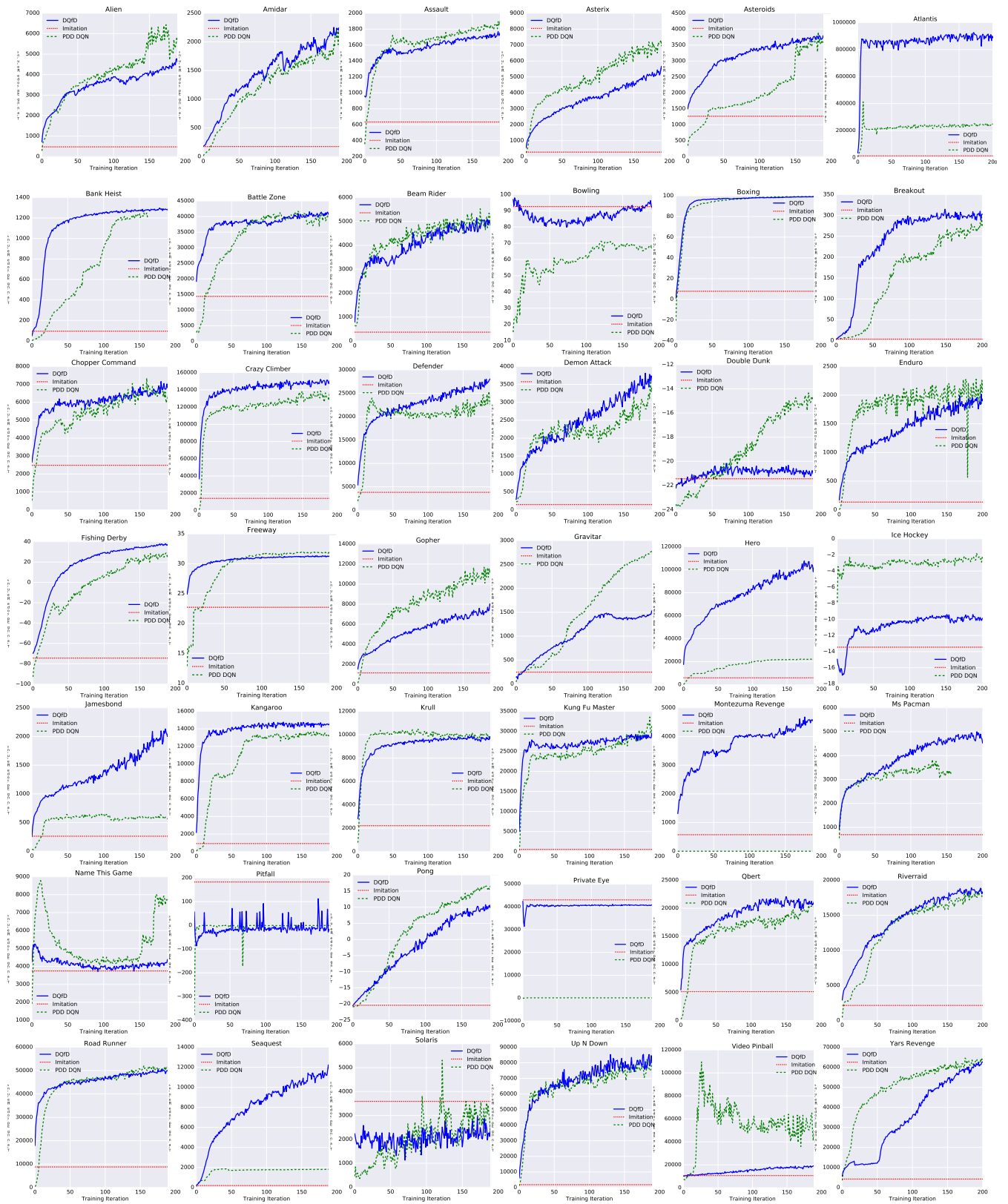


图 3: 三种算法在 42 款 Atari 游戏上的在线奖励, 4 次试验的平均值。每集以最多 30 个随机无操作动作开始。分数来自 Atari 游戏, 无论代理使用的奖励的内部表示如何。

Game	Worst Demo Score	Best Demo Score	Number Transitions	Number Episodes	DQfD Mean Score	PDD DQN Mean Score	Imitation Mean Score
Alien	9690	29160	19133	5	4737.5	6197.1	473.9
Amidar	1353	2341	16790	5	2325.0	2140.4	175.0
Assault	1168	2274	13224	5	1755.7	1880.9	634.4
Asterix	4500	18100	9525	5	5493.6	7566.2	279.9
Asteroids	14170	18100	22801	5	3796.4	3917.6	1267.3
Atlantis	10300	22400	17516	12	920213.9	303374.8	12736.6
Bank Heist	900	7465	32389	7	1280.2	1240.8	95.2
Battle Zone	35000	60000	9075	5	41708.2	41993.7	14402.4
Beam Rider	12594	19844	38665	4	5173.3	5401.4	365.9
Bowling	89	149	9991	5	97.0	71.0	92.6
Boxing	0	15	8438	5	99.1	99.3	7.8
Breakout	17	79	10475	9	308.1	275.0	3.5
Chopper Command	4700	11300	7710	5	6993.1	6973.8	2485.7
Crazy Climber	30600	61600	18937	5	151909.5	136828.2	14051.0
Defender	5150	18700	6421	5	27951.5	24558.8	3819.1
Demon Attack	1800	6190	17409	5	3848.8	3511.6	147.5
Double Dunk	-22	-14	11855	5	-20.4	-14.3	-21.4
Enduro	383	803	42058	5	1929.8	2199.6	134.8
Fishing Derby	-10	20	6388	4	38.4	28.6	-74.4
Freeway	30	32	10239	5	31.4	31.9	22.7
Gopher	2500	22520	38632	5	7810.3	12003.4	1142.6
Gravitar	2950	13400	15377	5	1685.1	2796.1	248.0
Hero	35155	99320	32907	5	105929.4	22290.1	5903.3
Ice Hockey	-4	1	17585	5	-9.6	-2.0	-13.5
James Bond	400	650	9050	5	2095.0	639.6	262.1
Kangaroo	12400	36300	20984	5	14681.5	13567.3	917.3
Krull	8040	13730	32581	5	9825.3	10344.4	2216.6
Kung Fu Master	8300	25920	12989	5	29132.0	32212.3	556.7
Montezuma's Revenge	32300	34900	17949	5	4638.4	0.1	576.3
Ms Pacman	31781	55021	21896	3	4695.7	3684.2	692.4
Name This Game	11350	19380	43571	5	5188.3	8716.8	3745.3
Pitfall	3662	47821	35347	5	57.3	0.0	182.8
Pong	-12	0	17719	3	10.7	16.7	-20.4
Private Eye	70375	74456	10899	5	42457.2	154.3	43047.8
Q-Bert	80700	99450	75472	5	21792.7	20693.7	5133.8
River Raid	17240	39710	46233	5	18735.4	18810.0	2148.5
Road Runner	8400	20200	5574	5	50199.6	51688.4	8794.9
Seaquest	56510	101120	57453	7	12361.6	1862.5	195.6
Solaris	2840	17840	28552	6	2616.8	4157.0	3589.6
Up N Down	6580	16080	10421	4	82555.0	82138.5	1816.7
Video Pinball	8409	32420	10051	5	19123.1	101339.6	10655.5
Yars' Revenge	48361	83523	21334	4	61575.7	63484.4	4225.8

Table 2: Best and worst human demonstration scores, the number of trials and transitions collected, and average score for each algorithm on the best 3 million step window, averaged over four seeds.

Game	Worst Demo	Best Demo	Number	Number	DQfD	PDD DQN	Imitation
	Score	Score	Transitions	Episodes	Mean Score	Mean Score	Mean Score
Alien	9690	29160	19133	5	4737.5	6197.1	473.9
Amidar	1353	2341	16790	5	2325.0	2140.4	175.0
Assault	1168	2274	13224	5	1755.7	1880.9	634.4
Asterix	4500	18100	9525	5	5493.6	7566.2	279.9
Asteroids	14170	18100	22801	5	3796.4	3917.6	1267.3
Atlantis	10300	22400	17516	12	920213.9	303374.8	12736.6
Bank Heist	900	7465	32389	7	1280.2	1240.8	95.2
Battle Zone	35000	60000	9075	5	41708.2	41993.7	14402.4
Beam Rider	12594	19844	38665	4	5173.3	5401.4	365.9
Bowling	89	149	9991	5	97.0	71.0	92.6
Boxing	0	15	8438	5	99.1	99.3	7.8
Breakout	17	79	10475	9	308.1	275.0	3.5
Chopper Command	4700	11300	7710	5	6993.1	6973.8	2485.7
Crazy Climber	30600	61600	18937	5	151909.5	136828.2	14051.0
Defender	5150	18700	6421	5	27951.5	24558.8	3819.1
Demon Attack	1800	6190	17409	5	3848.8	3511.6	147.5
Double Dunk	-22	-14	11855	5	-20.4	-14.3	-21.4
Enduro	383	803	42058	5	1929.8	2199.6	134.8
Fishing Derby	-10	20	6388	4	38.4	28.6	-74.4
Freeway	30	32	10239	5	31.4	31.9	22.7
Gopher	2500	22520	38632	5	7810.3	12003.4	1142.6
Gravitar	2950	13400	15377	5	1685.1	2796.1	248.0
Hero	35155	99320	32907	5	105929.4	22290.1	5903.3
Ice Hockey	-4	1	17585	5	-9.6	-2.0	-13.5
James Bond	400	650	9050	5	2095.0	639.6	262.1
Kangaroo	12400	36300	20984	5	14681.5	13567.3	917.3
Krull	8040	13730	32581	5	9825.3	10344.4	2216.6
Kung Fu Master	8300	25920	12989	5	29132.0	32212.3	556.7
Montezuma's Revenge	32300	34900	17949	5	4638.4	0.1	576.3
Ms Pacman	31781	55021	21896	3	4695.7	3684.2	692.4
Name This Game	11350	19380	43571	5	5188.3	8716.8	3745.3
Pitfall	3662	47821	35347	5	57.3	0.0	182.8
Pong	-12	0	17719	3	10.7	16.7	-20.4
Private Eye	70375	74456	10899	5	42457.2	154.3	43047.8
Q-Bert	80700	99450	75472	5	21792.7	20693.7	5133.8
River Raid	17240	39710	46233	5	18735.4	18810.0	2148.5
Road Runner	8400	20200	5574	5	50199.6	51688.4	8794.9
Seaquest	56510	101120	57453	7	12361.6	1862.5	195.6
Solaris	2840	17840	28552	6	2616.8	4157.0	3589.6
Up N Down	6580	16080	10421	4	82555.0	82138.5	1816.7
Video Pinball	8409	32420	10051	5	19123.1	101339.6	10655.5
Yars' Revenge	48361	83523	21334	4	61575.7	63484.4	4225.8

表 2: 最佳和最差的人类演示分数、收集的试验和转换次数以及每个算法在最佳 300 万步窗口上的平均分数（四个种子的平均值）。